Manual de Instalación y Configuración

Entorno de Desarrollo (Flutter)

Integrantes

- Adrián Pérez Jiménez
- Luis Daniel Suárez Escamilla
- Jonathan Emmanuel Lopez Morales
- Sayurid Bautista Cruz
- Jesús Domínguez Ramírez

28 de septiembre del 2026

Índice

1. Portada y Datos del Equipo
2. Introducción y Objetivo del Manual
   - 2.1. Introducción
   - 2.2. Objetivo General
   - 2.3. Objetivos Específicos
3. Módulo 1: Preparación del Entorno Base e IDE
   - 3.1. Instalación y Configuración de WSL (Ubuntu)
   - 3.2. Instalación y Configuración de Visual Studio Code (VSC)
   - 3.3. Instalación de Extensiones Requeridas en VSC
     - 3.3.1. Extensión WSL
     - 3.3.2. Pubspec Assist
     - 3.3.3. Error Lens
     - 3.3.4. Awesome Flutter Snippets
     - 3.3.5. Flutter & Dart
4. Módulo 2: Configuración del Entorno Móvil Flutter
   - 4.1. Instalación del SDK de Flutter y Variables de Entorno (PATH)
   - 4.2. Instalación de Android Studio y Android SDK Command-line Tools
   - 4.3. Aceptación de Licencias de Android (flutter doctor --android-licenses)
   - 4.4. Creación y Configuración del Dispositivo Virtual / Emulador (Pixel / Android)
   - 4.5. Diagnóstico y Verificación del Entorno (flutter doctor -v)
5. Módulo 3: Control de Versiones y Contenedorización
   - 5.1. Instalación y Verificación de Git y Configuración de GitHub
   - 5.2. Instalación de Docker Desktop y Soporte WSL2
   - 5.3. Verificación de Docker Engine y Docker Compose
   - 5.4. Archivo Integrador docker-compose.yml (Infraestructura Completa)
6. Módulo 4: Bases de Datos, Herramientas API y FastAPI
   - 6.1. Instalación y Comprobación de PostgreSQL
   - 6.2. Instalación y Comprobación de MongoDB
   - 6.3. Instalación del Cliente Postman
   - 6.4. Entorno Python, Instalación de FastAPI y Documentación Automática (Swagger UI)
7. Módulo 5: Servicios Backend, Keycloak y Conexión Flutter
   - 7.1. Instalación y Verificación de Node.js y Framework Express
   - 7.2. Despliegue e Instalación Base de Keycloak
   - 7.3. Configuración Básica de Cliente OIDC para App Móvil en Keycloak
   - 7.4. Configuración de Conexión Base en Flutter (pubspec.yaml y red 10.0.2.2)
8. ANEXO: Guía de despliegue rápido con Docker Compose
9. Conclusiones y Recomendaciones

2. Introducción y Objetivo del Manual

2.1. Introducción

El desarrollo de aplicaciones móviles modernas exige una integración fluida entre interfaces de usuario reactivas y una infraestructura backend escalable, modular y desacoplada. En este contexto, el presente manual establece la guía paso a paso para la instalación, configuración y validación de un entorno de desarrollo integral que combina Flutter como framework cliente y una arquitectura de microservicios y bases de datos contenedorizada sobre Docker.

Para garantizar la estabilidad y el máximo rendimiento del sistema, se ha implementado una estrategia de entorno híbrido:

- Entorno Nativo (Windows): Se destina a las herramientas de visualización y emulación de software móvil (Flutter SDK, Android Studio y el Emulador Pixel). Esto previene conflictos de aceleración por hardware (HAXM/KVM) y cuellos de botella en el puente de depuración de Android (ADB).
- Entorno Virtualizado/Contenedorizado (WSL2 + Docker): Se aprovecha el motor de Ubuntu en WSL2 para orquestar la infraestructura de persistencia (PostgreSQL, MongoDB), autenticación (Keycloak) y APIs backend (FastAPI, Node.js/Express).
- Esta estructura unificada permite a cada integrante del equipo trabajar sobre un conjunto estandarizado de herramientas, reduciendo a cero las discrepancias de entorno ("en mi máquina sí funciona") y agilizando el flujo de trabajo colaborativo.

2.2. Objetivo General

Proporcionar una guía técnica estandarizada y paso a paso para la instalación, configuración, orquestación y verificación de un entorno de desarrollo completo (móvil y backend), asegurando la correcta conectividad entre la aplicación móvil en Flutter y la infraestructura de servicios contenedorizada en Docker.

2.3. Objetivos Específicos

- **Establecer el Entorno Base y el IDE:** Configurar el subsistema de Linux para Windows (WSL2 Ubuntu) e integrar Visual Studio Code con las extensiones necesarias para el desarrollo remoto y análisis de código en Dart/Flutter.

- **Desplegar el Stack Móvil de Flutter:** Instalar de forma nativa el SDK de Flutter, configurar las herramientas CLI de Android Studio, aceptar licencias de desarrollo y crear un dispositivo virtual Pixel (AVD) verificado mediante flutter doctor -v.
- **Orquestar la Infraestructura Backend con Docker:** Configurar Docker Desktop con soporte WSL2 y ejecutar un archivo orquestador integrador (docker-compose.yml) que levante de manera simultánea PostgreSQL, MongoDB, Keycloak, FastAPI y Express.
- **Validar la Persistencia y APIs REST:** Comprobar la accesibilidad a los motores de base de datos y la disponibilidad de endpoints HTTP (incluyendo la documentación interactiva Swagger UI en FastAPI) utilizando clientes como Postman y el navegador web.
- **Asegurar la Conectividad Móvil-Backend:** Configurar la autenticación mediante un cliente OIDC en Keycloak e implementar el direccionamiento de red adecuado para permitir que la aplicación móvil desde el emulador consuma los servicios locales de Docker.

Módulo 1: Preparación del Entorno Base y de IDE

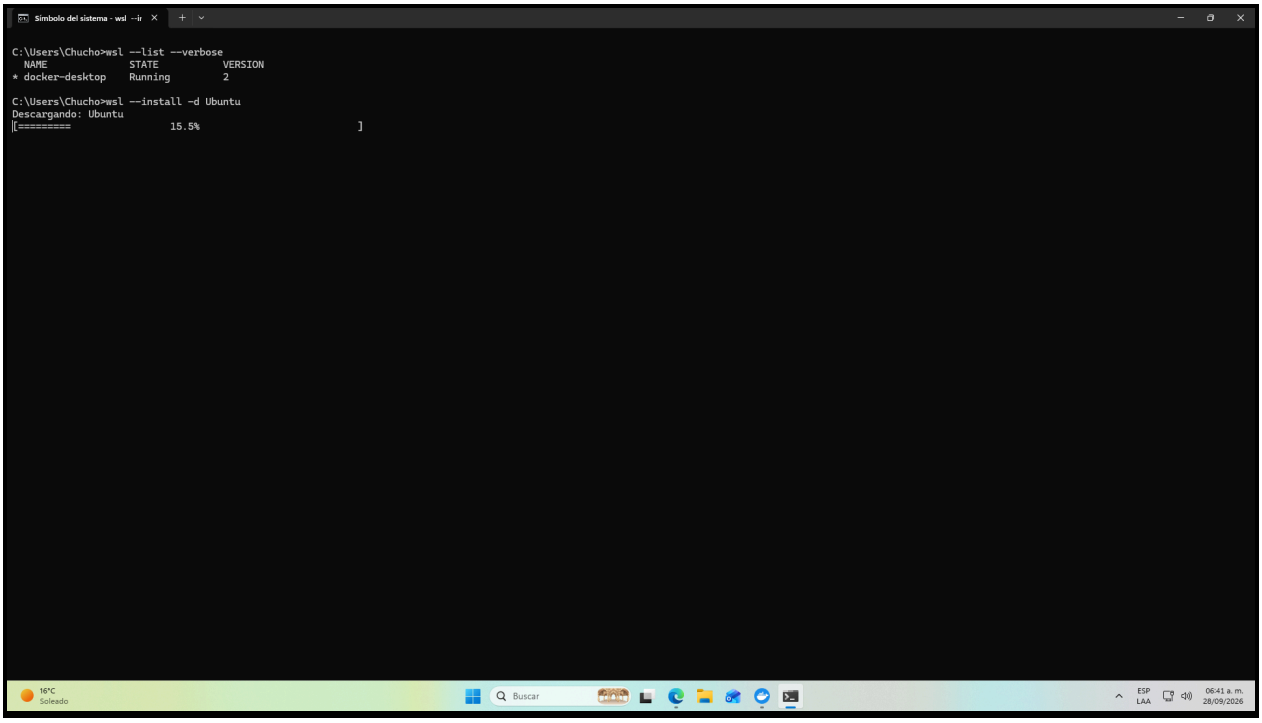
3.1 Instalación y Configuración de WSL (Ubuntu)

Pasos de Instalación

Haz clic derecho sobre el menú de Inicio de Windows y abre **Windows PowerShell (Administrador)** o **Terminal (Administrador)**.

1. Ejecuta el siguiente comando para descargar e instalar el Subsistema de Windows para Linux junto con la distribución de Ubuntu:
2. Bash

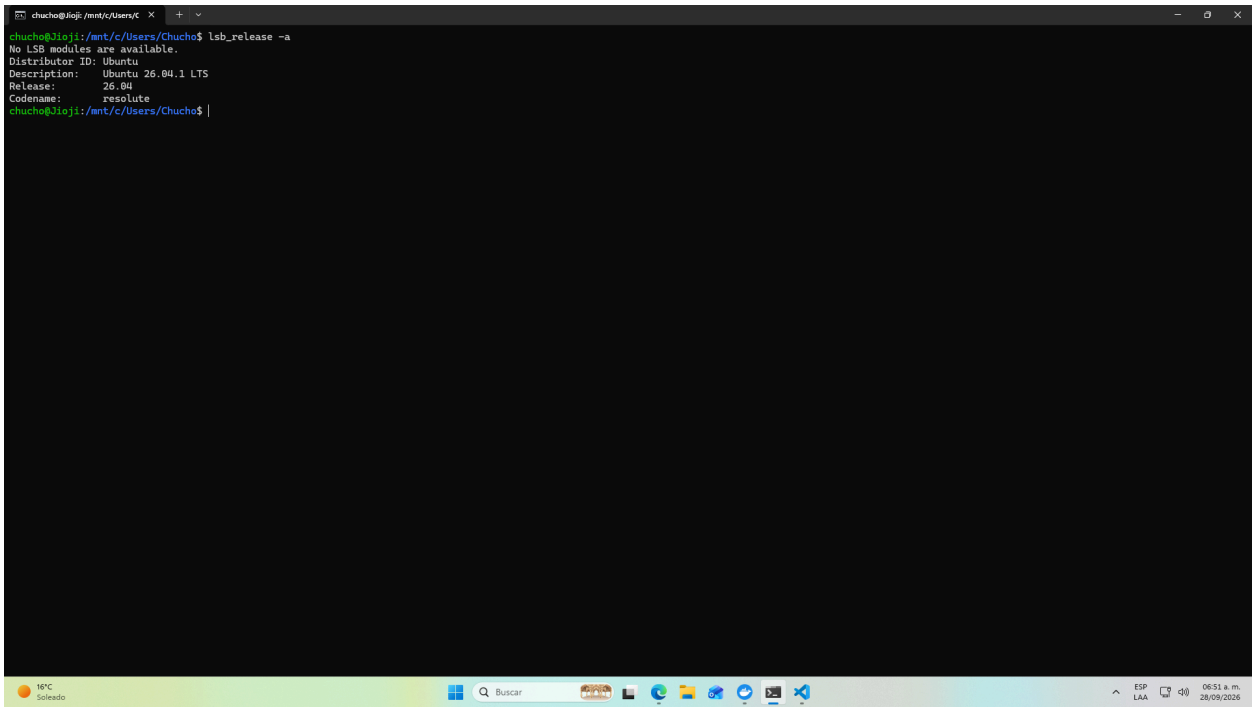
```
wsl --install -d Ubuntu
```



3. *Nota: Este comando habilita las características necesarias de WSL, descarga el kernel de Linux e instala Ubuntu automáticamente.*
4. Reinicia tu computadora si la terminal te lo solicita para aplicar los cambios.
5. Abre la aplicación **Ubuntu** desde el menú de inicio de Windows. La primera vez tardará unos minutos en descomprimirse.

6. Cuando se te solicite, ingresa un **nuevo nombre de usuario** (UNIX username) y presiona Enter.
7. Ingresa una **contraseña** (los caracteres no se verán mientras escribes por seguridad), presiona Enter y confírmala.
8. Para verificar rápidamente que la distribución está funcional, ejecuta el siguiente comando:

```
lsb_release -a
```

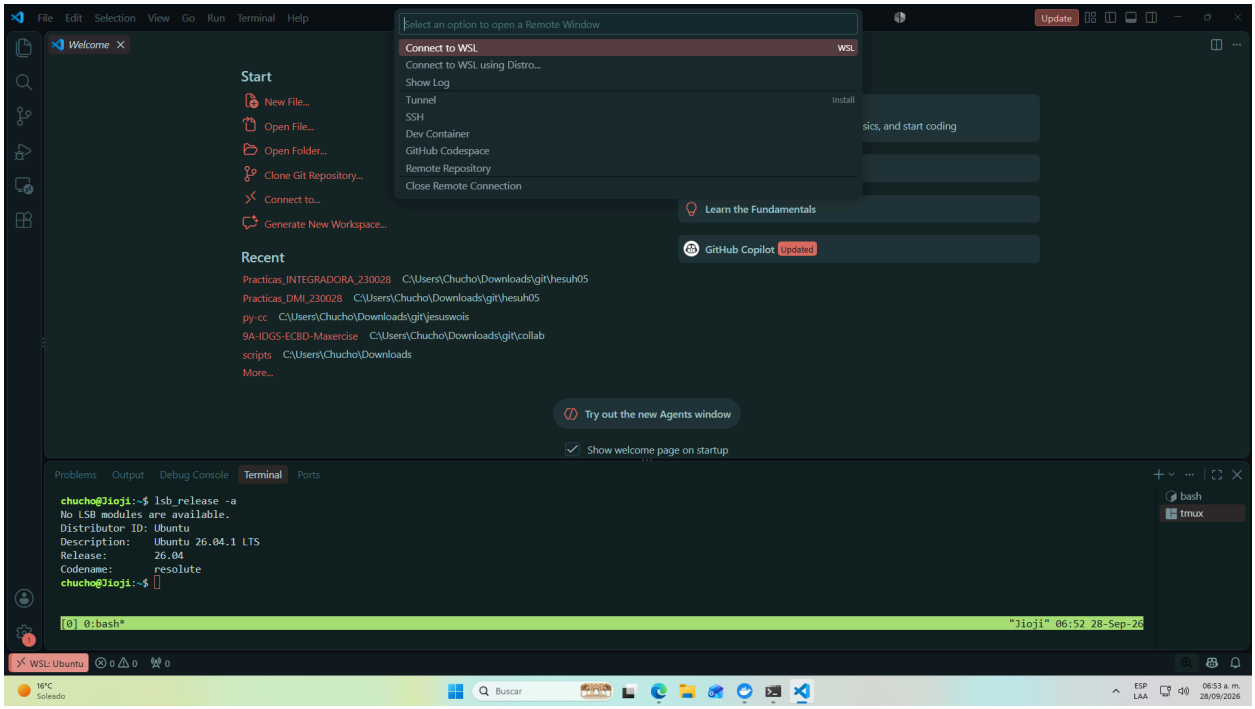


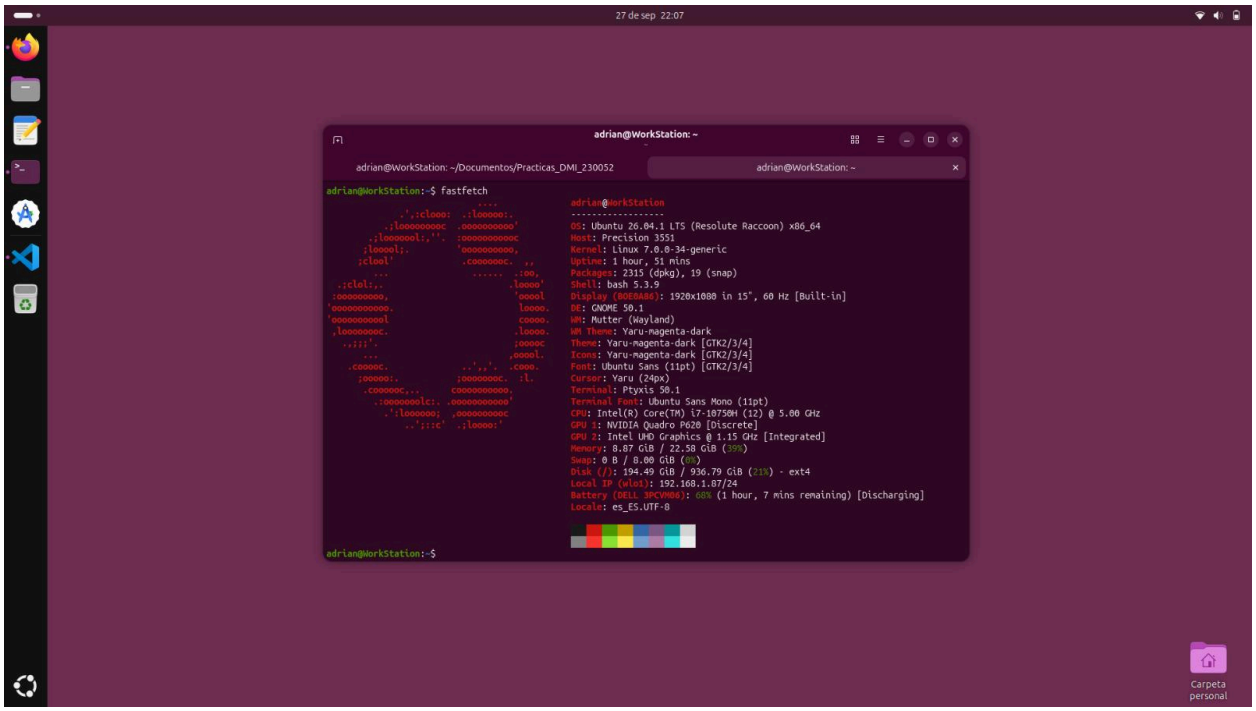
3.2. Instalación y Configuración de Visual Studio Code (VSC)

Descarga el instalador oficial para Windows desde code.visualstudio.com.

1. Ejecuta el archivo instalador descargado.
2. Acepta el acuerdo de licencia y presiona **Siguiente**.

3. En la ventana de configuración de tareas adicionales, asegúrate de marcar la opción **"Agregar a PATH (disponible después de reiniciar)"**. Es muy recomendable marcar también "Agregar la acción 'Abrir con Code'".
4. Haz clic en **Instalar** y espera a que el asistente finalice.
5. Finaliza el proceso y lanza Visual Studio Code.





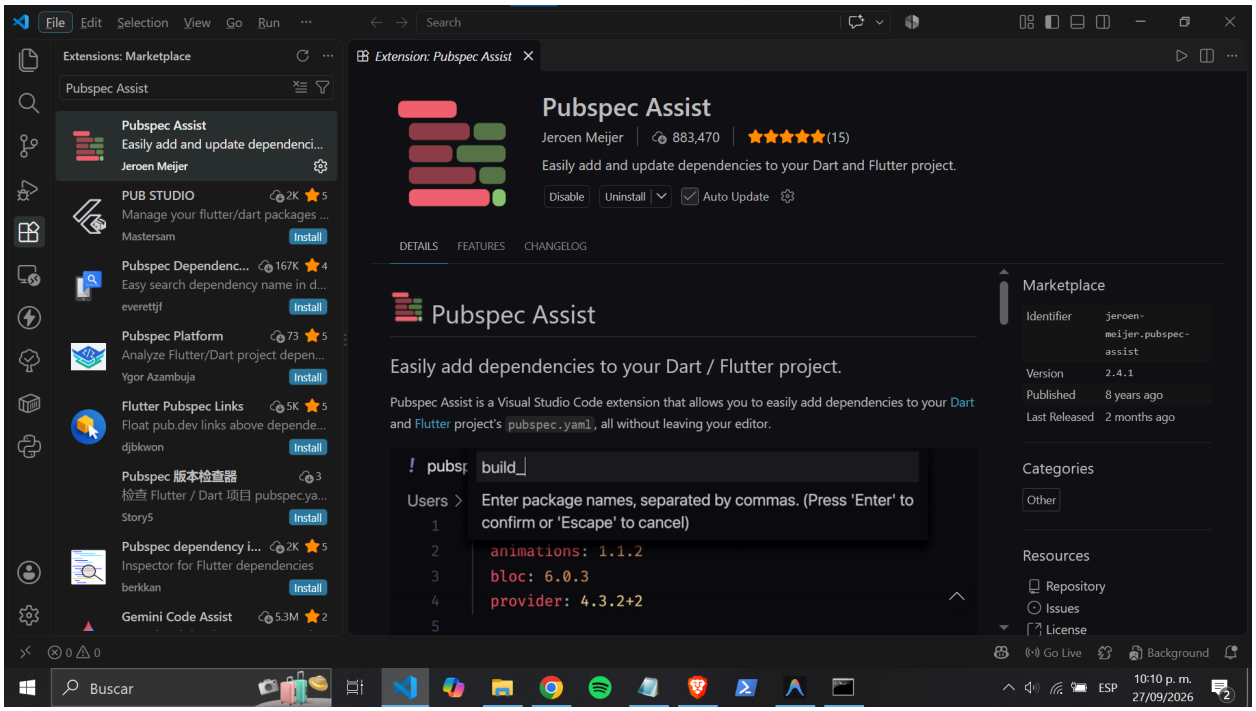
3.3. Instalación de Extensiones Requeridas en VSC

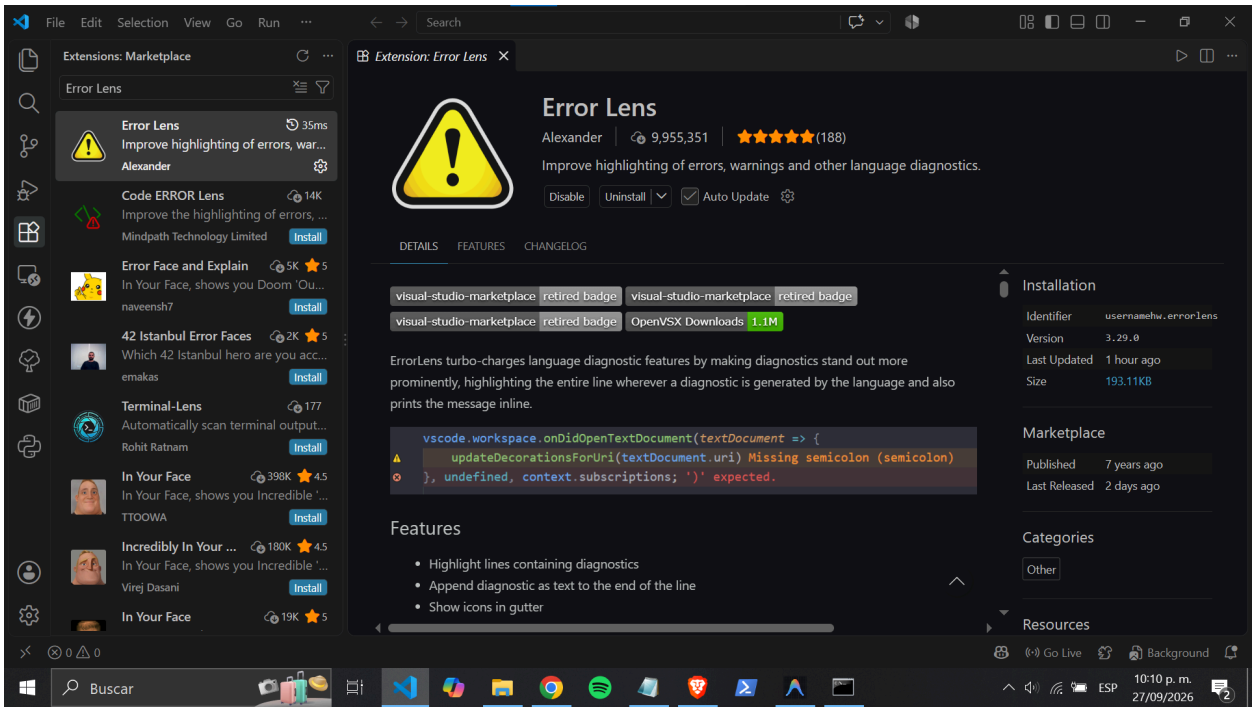
Abre Visual Studio Code.

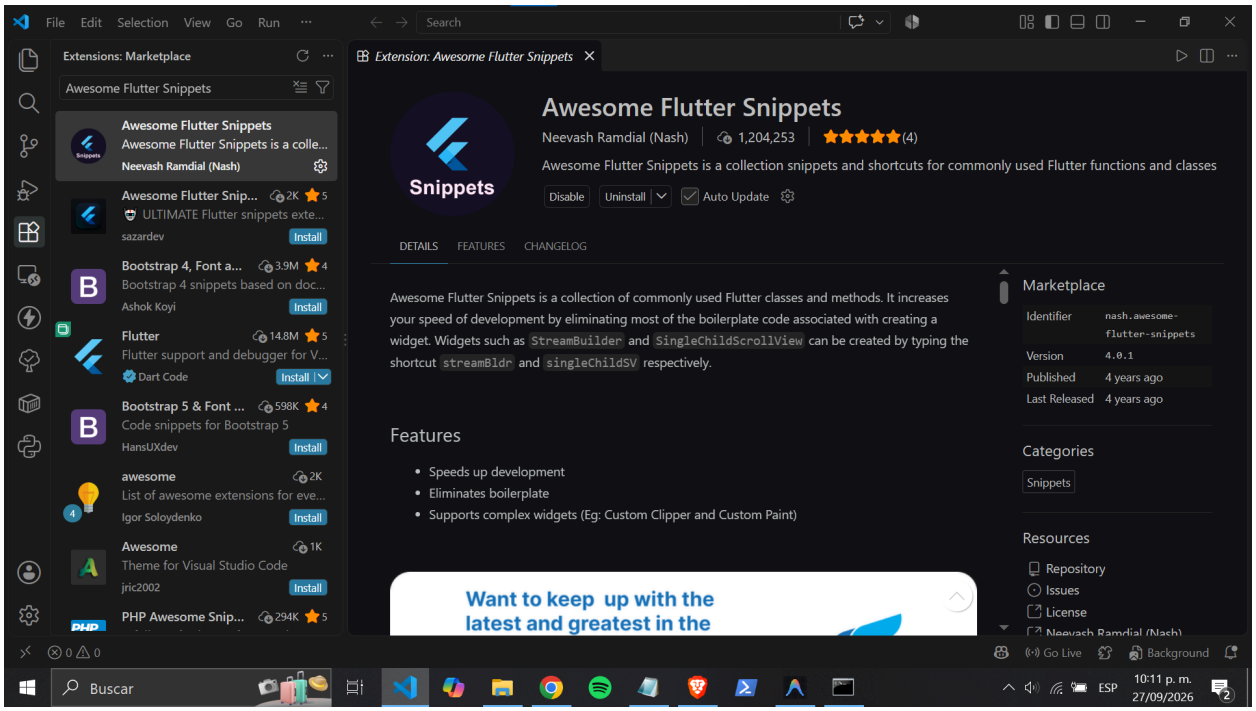
1. Abre el panel de extensiones presionando `Ctrl + Shift + X` o haciendo clic en el ícono de bloques en la barra lateral izquierda.
2. En la barra de búsqueda, escribe el nombre de las siguientes extensiones y haz clic en **Instalar** en cada una de ellas:
    - **WSL**: Permite abrir carpetas del entorno de Linux directamente en VS Code en Windows.
    - **Pubspec Assist**: Facilita la búsqueda y adición automática de dependencias al archivo `pubspec.yaml` de Flutter.
    - **Error Lens**: Resalta los errores, advertencias y sugerencias de código directamente en la línea correspondiente, mejorando la visualización.
    - **Awesome Flutter Snippets**: Proporciona atajos y fragmentos de código (snippets) para agilizar la escritura de componentes de interfaz y clases.

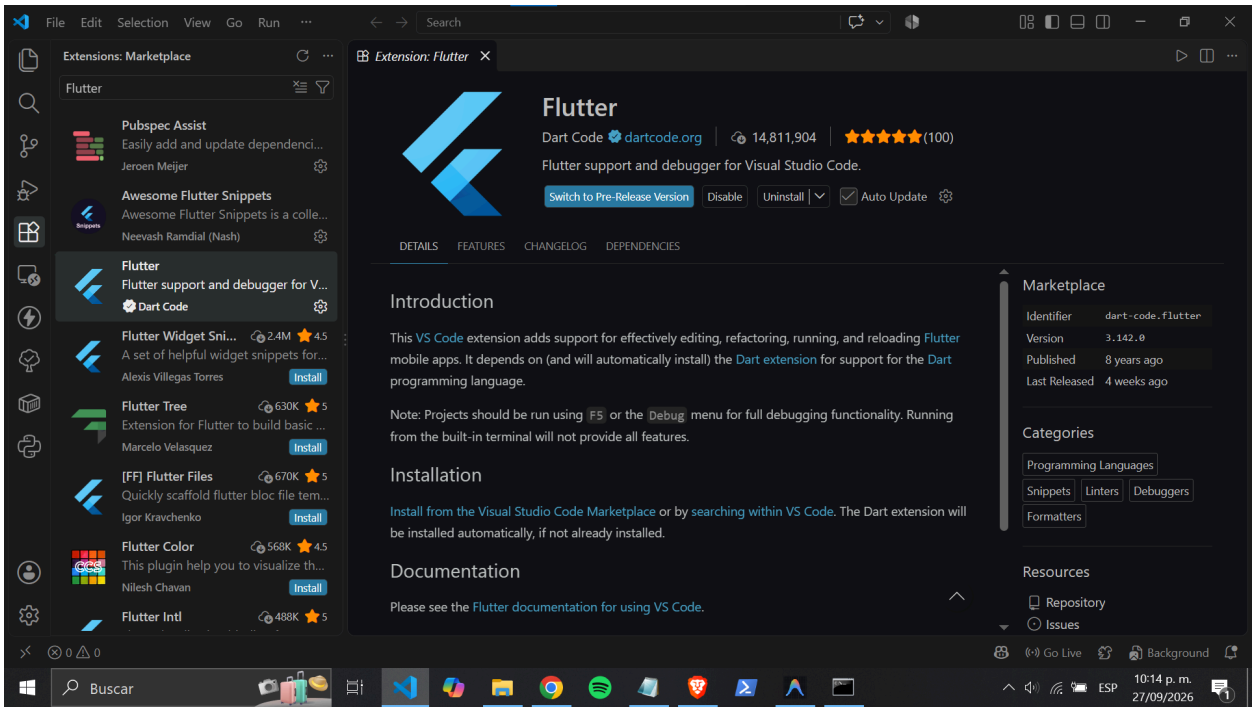
- **Flutter**: Proporciona soporte completo para desarrollo móvil, herramientas de depuración y refactorización. (*Nota: Su instalación incluye automáticamente la extensión de **Dart**).*




Extensions: Marketplace
Pubspec Assist
Extension: Pubspec Assist
Pubspec Assist
Jeroen Meijer
883,470
(15)
Easily add and update dependencies to your Dart and Flutter project.
Disable
Uninstall
Auto Update
DETAILS
FEATURES
CHANGELOG
Pubspec Assist
Easily add dependencies to your Dart / Flutter project.
Pubspec Assist is a Visual Studio Code extension that allows you to easily add dependencies to your Dart and Flutter project's pubspec.yaml, all without leaving your editor.
build_
Enter package names, separated by commas. (Press 'Enter' to confirm or 'Escape' to cancel)
animations: 1.1.2
bloc: 6.0.3
provider: 4.3.2+2
Marketplace
Identifier
jeroen-meijer.pubspec-assist
Version
2.4.1
Published
8 years ago
Last Released
2 months ago
Categories
Other
Resources
Repository
Issues
License
Buscar
ESP
10:10 p. m.
27/09/2026


Extensions: Marketplace
Error Lens
Extension: Error Lens
Error Lens
Alexander
9,955,351
(188)
Improve highlighting of errors, warnings and other language diagnostics.
Disable
Uninstall
Auto Update
DETAILS
FEATURES
CHANGELOG
visual-studio-marketplace retired badge
visual-studio-marketplace retired badge
visual-studio-marketplace retired badge
OpenVSX Downloads 1.1M
ErrorLens turbo-charges language diagnostic features by making diagnostics stand out more prominently, highlighting the entire line wherever a diagnostic is generated by the language and also prints the message inline.
vscode.workspace.onDidOpenTextDocument(textDocument => {
updateDecorationsForUri(textDocument.uri) Missing semicolon (semicolon)
}, undefined, context.subscriptions; ')' expected.
Features
Highlight lines containing diagnostics
Append diagnostic as text to the end of the line
Show icons in gutter
Installation
Identifier
usernamehw.errorlens
Version
3.29.0
Last Updated
1 hour ago
Size
193.11KB
Marketplace
Published
7 years ago
Last Released
2 days ago
Categories
Other
Resources
Buscar
ESP
10:10 p. m.
27/09/2026


Extensions: Marketplace
Awesome Flutter Snippets
Extension: Awesome Flutter Snippets
Awesome Flutter Snippets
Neevash Ramdial (Nash) | 1,204,253 | (4)
Awesome Flutter Snippets is a collection snippets and shortcuts for commonly used Flutter functions and classes
Disable | Uninstall | Auto Update
DETAILS | FEATURES | CHANGELOG
Awesome Flutter Snippets is a collection of commonly used Flutter classes and methods. It increases your speed of development by eliminating most of the boilerplate code associated with creating a widget. Widgets such as StreamBuilder and SingleChildScrollView can be created by typing the shortcut streamBldr and singleChildSV respectively.
Features
Speeds up development
Eliminates boilerplate
Supports complex widgets (Eg: Custom Clipper and Custom Paint)
Want to keep up with the latest and greatest in the
Marketplace
Identifier nash.awesome-flutter-snippets
Version 4.0.1
Published 4 years ago
Last Released 4 years ago
Categories
Snippets
Resources
Repository
Issues
License
Buscar
ESP
10:11 p. m.
27/09/2026


Extensions: Marketplace
Flutter
Extension: Flutter
Flutter
Dart Code dartcode.org | 14,811,904 | (100)
Flutter support and debugger for Visual Studio Code.
Switch to Pre-Release Version | Disable | Uninstall | Auto Update
DETAILS | FEATURES | CHANGELOG | DEPENDENCIES
Introduction
This VS Code extension adds support for effectively editing, refactoring, running, and reloading Flutter mobile apps. It depends on (and will automatically install) the Dart extension for support for the Dart programming language.
Note: Projects should be run using F5 or the Debug menu for full debugging functionality. Running from the built-in terminal will not provide all features.
Installation
Install from the Visual Studio Code Marketplace or by searching within VS Code. The Dart extension will be installed automatically, if not already installed.
Documentation
Please see the Flutter documentation for using VS Code.
Marketplace
Identifier dart-code.flutter
Version 3.142.0
Published 8 years ago
Last Released 4 weeks ago
Categories
Programming Languages
Snippets | Linters | Debuggers
Formatters
Resources
Repository
Issues
Buscar
ESP
10:14 p. m.
27/09/2026

4. Módulo 2: Entorno Flutter y Emulador Mobile

En este módulo se describe el proceso de instalación y configuración del entorno necesario para desarrollar aplicaciones móviles utilizando **Flutter** y **Android Studio** en Windows. Se incluye la configuración del SDK de Flutter, las herramientas de desarrollo para Android, la aceptación de licencias, la creación de un dispositivo virtual y la verificación final del entorno.

4.1. Instalación del SDK de Flutter y Variables de Entorno (PATH)

Para comenzar a desarrollar aplicaciones con Flutter es necesario instalar el **Flutter SDK** y agregar su carpeta de ejecución a las variables de entorno de Windows.

Instalación del SDK de Flutter

1. Ingresar al sitio oficial de Flutter desde el navegador.
2. Descargar la versión del **Flutter SDK para Windows**.
3. Una vez finalizada la descarga, localizar el archivo comprimido de Flutter.
4. Extraer el contenido en una carpeta de fácil acceso. Por ejemplo:

   ```
   C:\development\flutter
   ```

Es recomendable evitar carpetas que requieran permisos administrativos o que tengan rutas demasiado complejas.

5. Verificar que dentro de la carpeta extraída se encuentre el directorio:

   ```
   C:\development\flutter
   ```

Configuración de la variable PATH

Para poder ejecutar los comandos de Flutter desde cualquier terminal de Windows se debe agregar la carpeta `bin` a la variable de entorno `Path`.

1. Abrir el menú **Inicio de Windows**.
2. Buscar: *Editar las variables de entorno del sistema*
3. Abrir la opción correspondiente y seleccionar **Variables de entorno**.
4. Dentro de la sección de variables de usuario, seleccionar la variable: *Path*
5. Presionar **Editar**.
6. Seleccionar **Nuevo**.

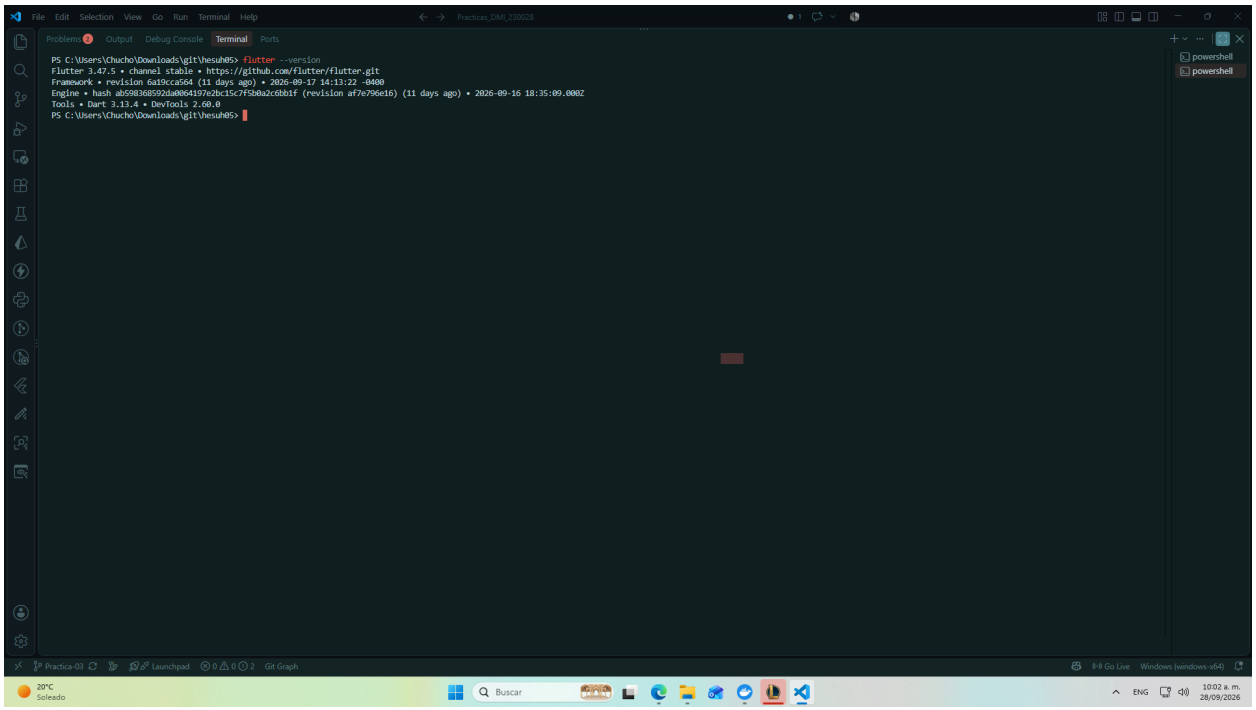
7. Agregar la ruta de la carpeta `bin` de Flutter. Por ejemplo:

   ```
   C:\development\flutter\bin
   ```

8. Presionar **Aceptar** para guardar todos los cambios.
9. Cerrar y volver a abrir CMD o PowerShell para que la nueva configuración sea reconocida.
10. Comprobar la instalación ejecutando:

    ```
    flutter --version
    ```

Si la instalación fue realizada correctamente, la terminal mostrará información sobre la versión instalada de Flutter y Dart.



4.2. Instalación de Android Studio y Android SDK Command-line Tools

Flutter necesita las herramientas de desarrollo de Android para compilar y ejecutar aplicaciones en dispositivos Android. Para ello se utilizará **Android Studio instalado de manera nativa en Windows**.

Instalación de Android Studio

1. Descargar el instalador de Android Studio desde su sitio oficial.

2. Ejecutar el archivo descargado.
3. Seguir el asistente de instalación.
4. Durante el proceso, mantener seleccionados los componentes principales:
    - Android Studio.
    - Android SDK.
    - Android SDK Platform.
    - Android Virtual Device.
5. Finalizar la instalación y ejecutar Android Studio.
6. En la primera ejecución, completar el asistente inicial para descargar los componentes requeridos.

Configuración del Android SDK

1. Abrir Android Studio.
2. Acceder a la configuración del **SDK Manager**.
3. Dentro de la pestaña **SDK Platforms**, verificar que exista al menos una versión reciente del SDK de Android instalada.
4. Posteriormente, ingresar a la pestaña:

SDK Tools

5. Verificar o seleccionar las siguientes herramientas:

- Android SDK Build-Tools.
- Android SDK Platform-Tools.
- Android SDK Command-line Tools (latest).
- Android Emulator.

6. Presionar **Apply**.
7. Aceptar los cambios y esperar a que Android Studio descargue e instale los componentes seleccionados.

Las **Android SDK Command-line Tools** son especialmente importantes debido a que Flutter las utiliza para realizar tareas como la aceptación de licencias y la comprobación del entorno de desarrollo.

4.3. Aceptación de Licencias de Android (flutter doctor --android-licenses)

Después de instalar Android Studio y las herramientas del SDK, es necesario aceptar las licencias de Android para que Flutter pueda utilizar correctamente el SDK.

1. Abrir CMD o PowerShell.

2. Ejecutar el siguiente comando:

   ```
   flutter doctor --android-licenses
   ```

3. La terminal mostrará diferentes licencias y solicitará confirmación para aceptarlas.
4. Escribir: *"Y"* *y presionar* **Enter** cada vez que se solicite aceptar una licencia.
5. Repetir el procedimiento hasta finalizar el proceso.

Al terminar correctamente, Flutter indicará que las licencias del SDK de Android fueron aceptadas.

Este procedimiento permite que Flutter tenga autorización para utilizar las herramientas necesarias para compilar y ejecutar aplicaciones Android.

En caso de que el comando indique que no encuentra `sdk manager`, se debe regresar al **SDK Manager** de Android Studio y comprobar que se encuentre instalado:

Android SDK Command-line Tools (latest)

4.4. Creación y Configuración del Dispositivo Virtual / Emulador (Pixel / Android)

Para ejecutar aplicaciones Flutter sin utilizar un teléfono físico se puede crear un dispositivo Android virtual mediante el **Device Manager** incluido en Android Studio.

Creación del dispositivo virtual

1. Abrir Android Studio.
2. Acceder a: *Device Manager*
3. Seleccionar la opción para crear un nuevo dispositivo virtual.
4. Elegir la categoría: *Phone*
5. Seleccionar un dispositivo de la familia **Google Pixel**.

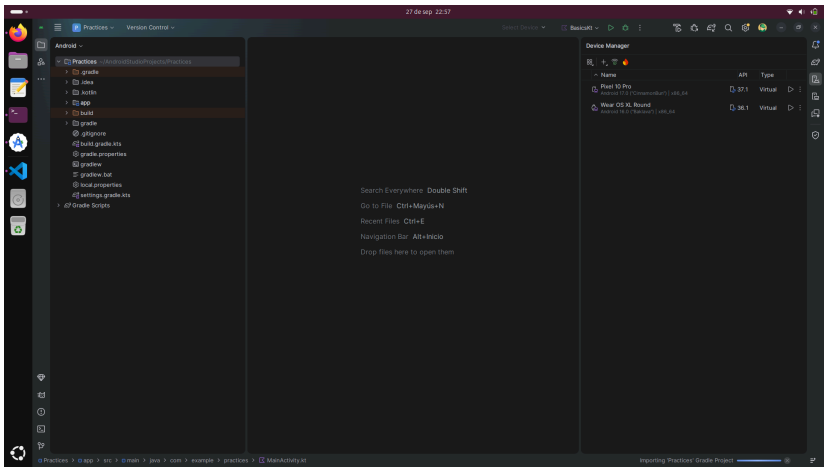
Se puede utilizar **Pixel 10 Pro** si se encuentra disponible. En caso de que dicho modelo no aparezca, se debe seleccionar el modelo Pixel más reciente disponible, por ejemplo:

Pixel 9 Pro o Pixel 8 Pro

6. Presionar **Next**.
7. Seleccionar una imagen del sistema Android disponible.
8. Si la imagen del sistema todavía no se encuentra instalada, seleccionar la opción **Download** y esperar a que termine la descarga.

9. Seleccionar la imagen instalada y continuar con **Next**.
10. Verificar la configuración del dispositivo virtual.
11. Asignar un nombre al dispositivo si se desea.
12. Presionar **Finish** para crear el emulador.

Una vez terminado el proceso, el nuevo dispositivo aparecerá dentro del **Device Manager**.



La presencia del dispositivo dentro del Device Manager confirma que el entorno virtual para ejecutar aplicaciones Android fue creado correctamente.

Ejecución del emulador

Para iniciar el dispositivo virtual:

1. Localizar el dispositivo creado dentro del Device Manager.
2. Presionar el botón de ejecución correspondiente.
3. Esperar a que Android complete el proceso de inicio.
4. Una vez que el teléfono virtual se encuentre encendido, abrir CMD o PowerShell.
5. Ejecutar:

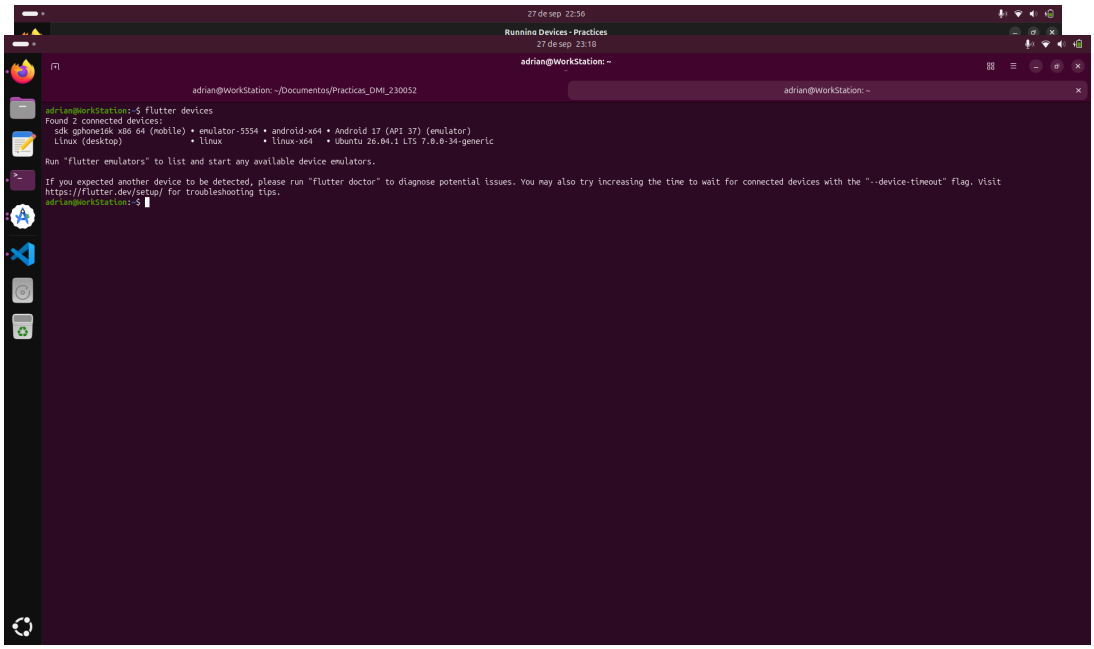
```
flutter devices
```

Flutter deberá mostrar el emulador Android dentro de la lista de dispositivos disponibles.

Ejemplo de la información que puede aparecer:

```
Found devices:
```

```
Android SDK built for x86 • emulator-5554 • android-x86 • Android
```



Esta comprobación demuestra que Flutter reconoce correctamente el dispositivo virtual y que puede utilizarlo posteriormente para ejecutar aplicaciones.

4.5. Diagnóstico y Verificación del Entorno (flutter doctor -v)

Finalmente, se debe verificar que todos los componentes necesarios para el desarrollo móvil se encuentren correctamente configurados.

Flutter proporciona la herramienta `flutter doctor`, encargada de analizar automáticamente el entorno instalado.

Abrir CMD o PowerShell y ejecutar:

```
flutter doctor -v
```

El parámetro: `-v` permite mostrar información detallada sobre cada uno de los componentes analizados.

Flutter realizará la comprobación de elementos como:

- Flutter SDK.
- Dart SDK.
- Android SDK.

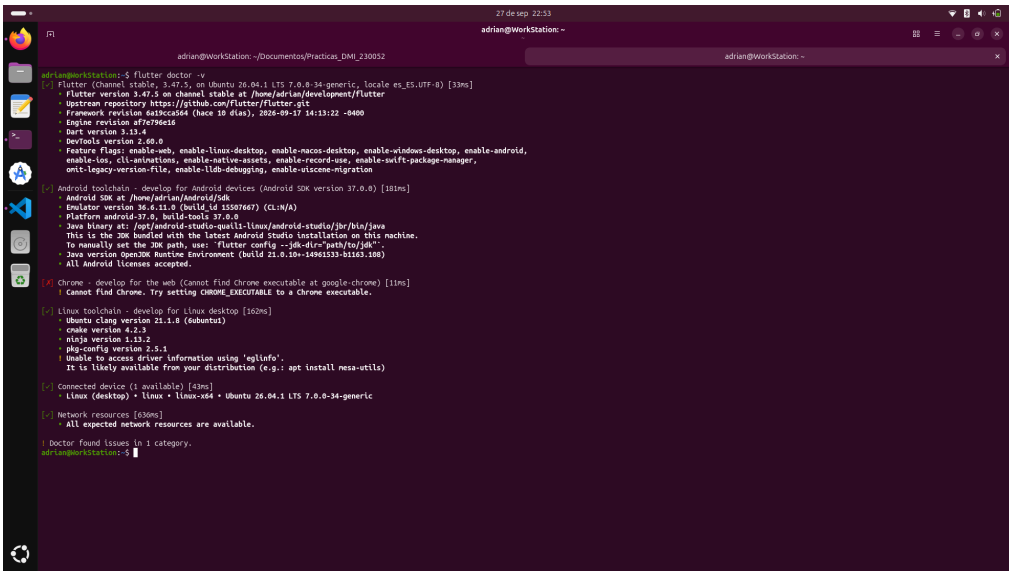
- Android SDK Command-line Tools.
- Licencias de Android.
- Dispositivos conectados o emuladores disponibles.

Cuando un componente está correctamente instalado se muestra mediante el símbolo:

[√]

En caso de existir algún problema, Flutter puede mostrar símbolos de advertencia o error acompañados de información sobre la configuración que debe corregirse.

Para este entorno, los principales elementos relacionados con el desarrollo Android deben encontrarse reconocidos correctamente.



Esta comprobación representa la verificación final del módulo, ya que permite confirmar que **Flutter reconoce Android Studio, el Android SDK, las herramientas de línea de comandos y el dispositivo disponible para ejecutar aplicaciones**.

También se puede realizar una comprobación adicional mediante:

flutter devices

Con ello se valida que Flutter dispone de al menos un dispositivo Android en el cual ejecutar el proyecto.

Una vez realizadas estas configuraciones, el entorno queda preparado para crear, compilar y ejecutar aplicaciones Flutter para Android desde Windows.

5. Módulo 3: Control de Versiones y Contenedorización

5.1 Instalación y Verificación de Git y Configuración de GitHub

Pasos de Instalación

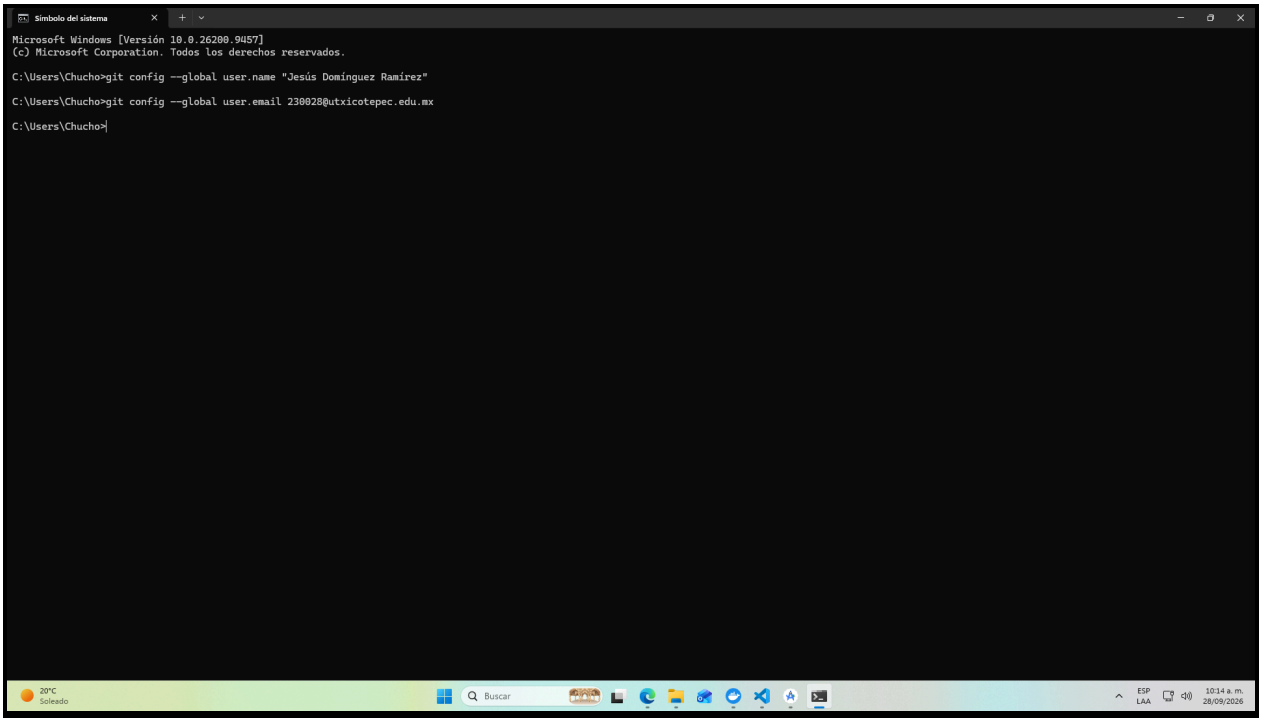
1. Descargar el instalador oficial de Git para Windows desde el sitio web https://git-scm.com/download/win.
2. Ejecutar el archivo .exe descargado y avanzar manteniendo las opciones por defecto recomendadas, asegurando que la casilla "Git from the command line and also from 3rd-party software" esté marcada.
3. Finalizar la instalación.

Configuración Global de Credenciales

1. Abrir la terminal de Windows (PowerShell, CMD o Git Bash) y ejecutar los siguientes comandos reemplazando con los datos correspondientes de la cuenta de GitHub:

   ```
   git config --global user.name "Tu Nombre Completo"
   ```

   ```
   git config --global user.email "tu_correo@ejemplo.com"
   ```

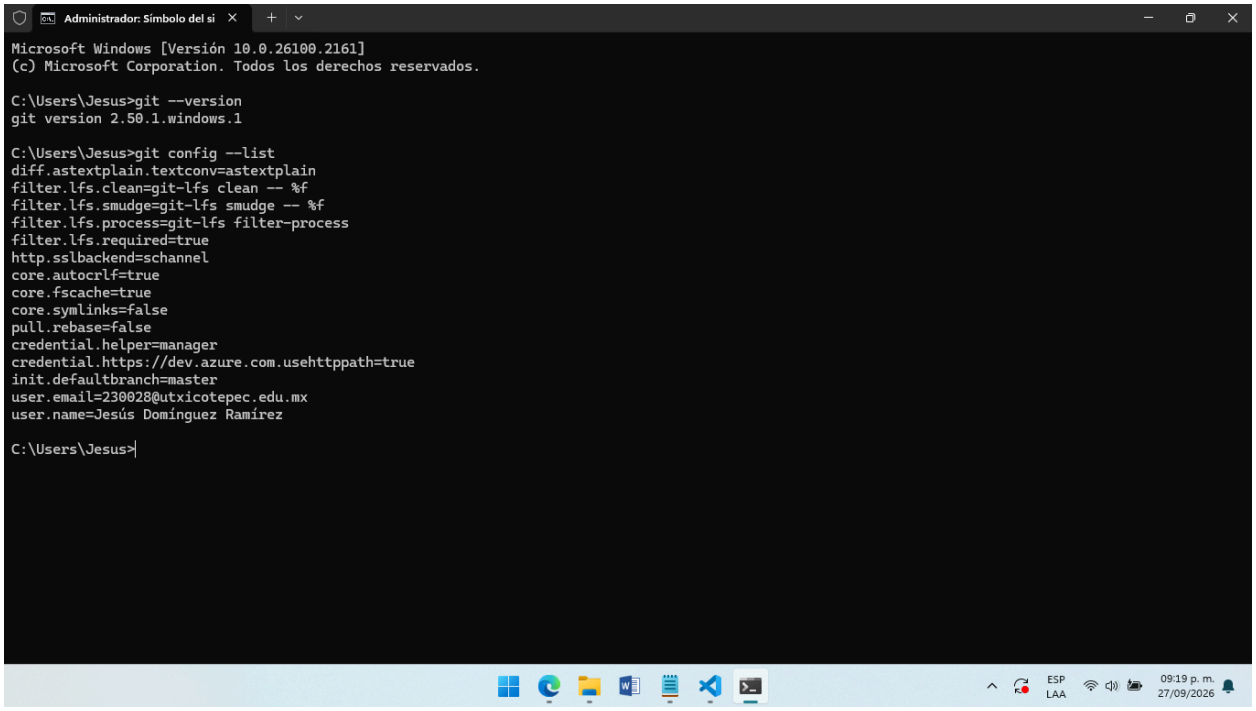


Comprobación Rápida

Ejecutar la consola:

```
git --version
```

```
git config --list
```



Descripción: Valida que Git está instalado, accesible desde las variables del entorno del sistema (PATH) y con las credenciales de usuario configuradas correctamente.

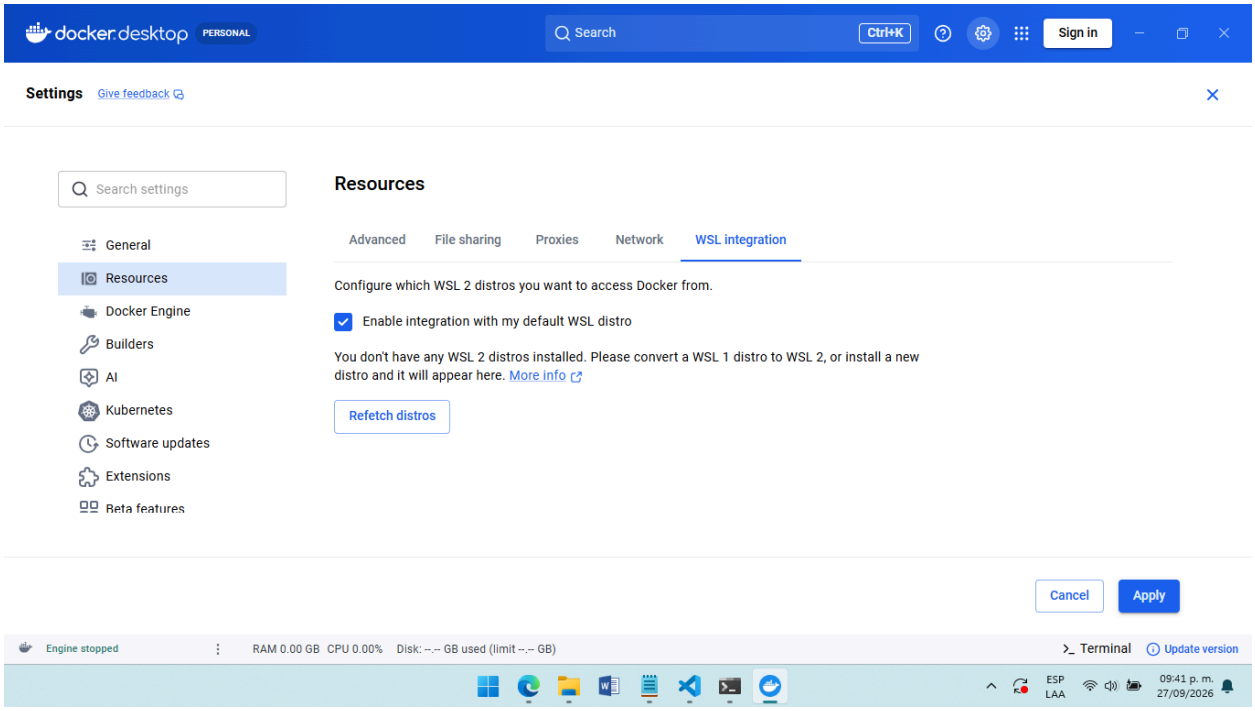
5.2 Instalación de Docker Desktop y Soporte WSL2:

Pasos de Instalación

1. Descargar el ejecutable de Docker Desktop para Windows desde https://www.docker.com/products/docker-desktop/.
2. Iniciar el instalador. En la pantalla de configuración, asegurarse de tener seleccionada la opción "Use WSL 2 instead of Hyper-V (recommended)".
3. Una vez completada la instalación, reiniciar el equipo si el instalador lo solicita.

Habilitación de la Integración con WSL2

1. Abrir la aplicación Docker Desktop.
2. Ir al icono de engranaje (Settings) en la esquina superior derecha.
3. Navegar a Resources > WSL Integration.
4. Activar la casilla "Enable integration with my default WSL distro" y asegurar que la distribución Ubuntu esté habilitada.
5. Hacer clic en Apply & restart.



5.3 Verificación de Docker Engine y Docker Compose:

Comprobación en Consola

Abrir PowerShell o la terminal de WSL (Ubuntu) y ejecutar los siguientes comandos para confirmar que el motor de contenedores y el plugin Compose están activos:

Comprobar la versión de Docker Engine

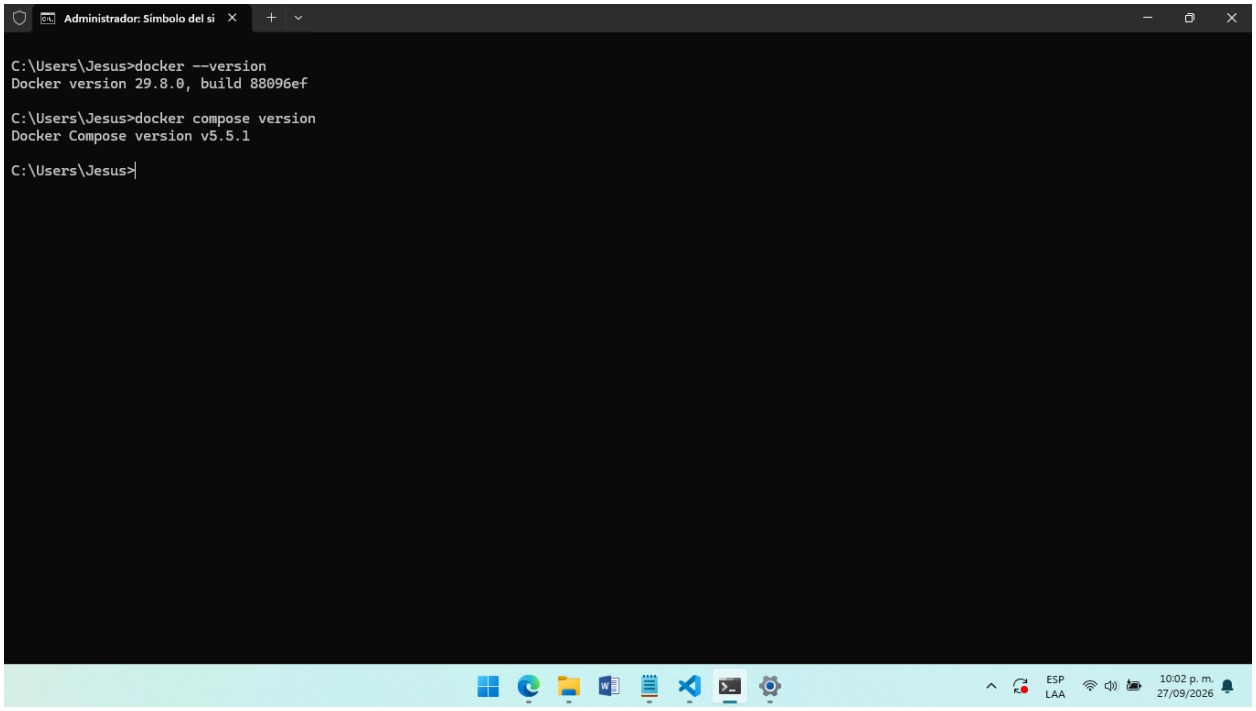
```
docker --version
```

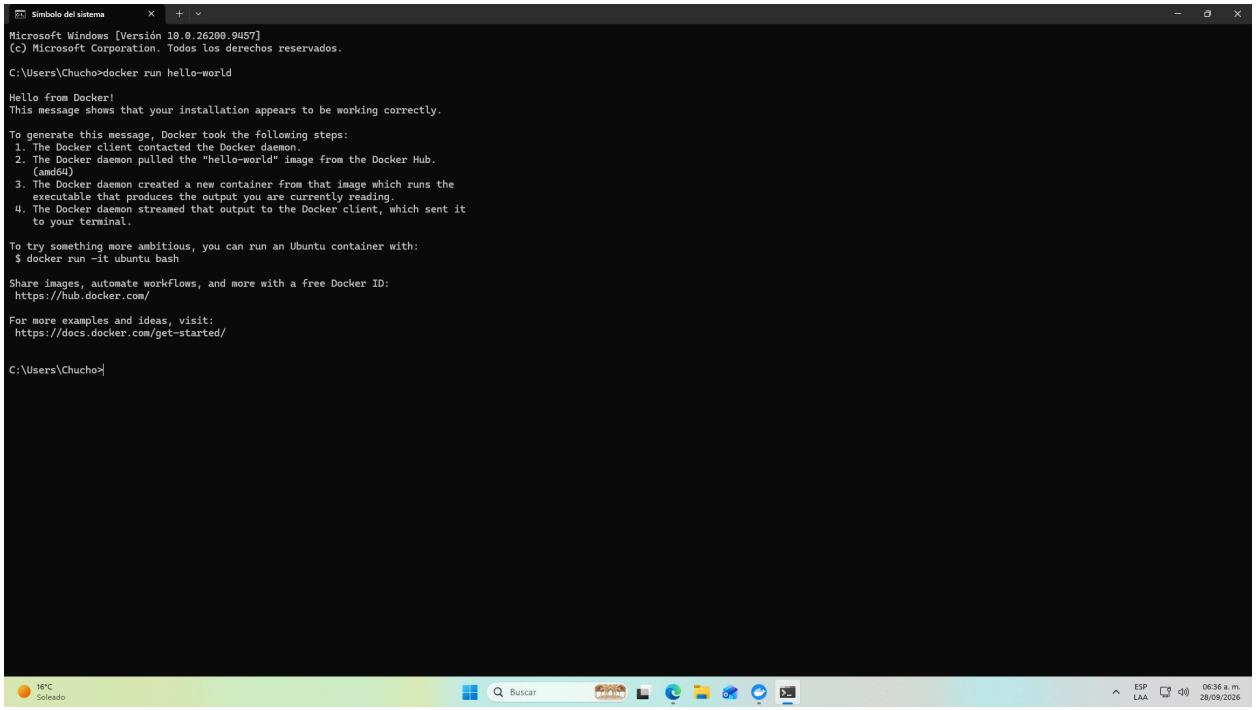
Comprobar la versión de Docker Compose (V2)

```
docker compose version
```

Probar la ejecución de un contenedor de prueba

```
docker run hello-world
```





5.4 Archivo Integrador docker-compose.yml (Infraestructura Completa):

Para desplegar la infraestructura base que requieren los módulos del manual en un solo paso, se utiliza un archivo orquestador de Docker Compose (compatible con la especificación V2).

Creación del Archivo

Crear un archivo llamado docker-compose.yml en la raíz del directorio del proyecto con el siguiente contenido exacto:

```
services:
  postgres:
    image: postgres:15
    container_name: manual_postgres
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password123
      POSTGRES_DB: dev_db
    ports:
      - "5432:5432"

  mongo:
    image: mongo:latest
    container_name: manual_mongo
    ports:
      - "27017:27017"

  keycloak:
    image: quay.io/keycloak/keycloak:latest
    container_name: manual_keycloak
    command: start-dev
    environment:
```

```
    KEYCLOAK_ADMIN: admin
    KEYCLOAK_ADMIN_PASSWORD: admin
  ports:
    - "8080:8080"

fastapi_app:
  image: tiangolo/uvicorn-gunicorn-fastapi:python3.10
  container_name: manual_fastapi
  ports:
    - "8000:8000"

express_app:
  image: node:18-alpine
  container_name: manual_express
  command: node -e "require('http').createServer((_, r) => r.end('Express Ready')).listen(3000)"
  ports:
    - "3000:3000"
```

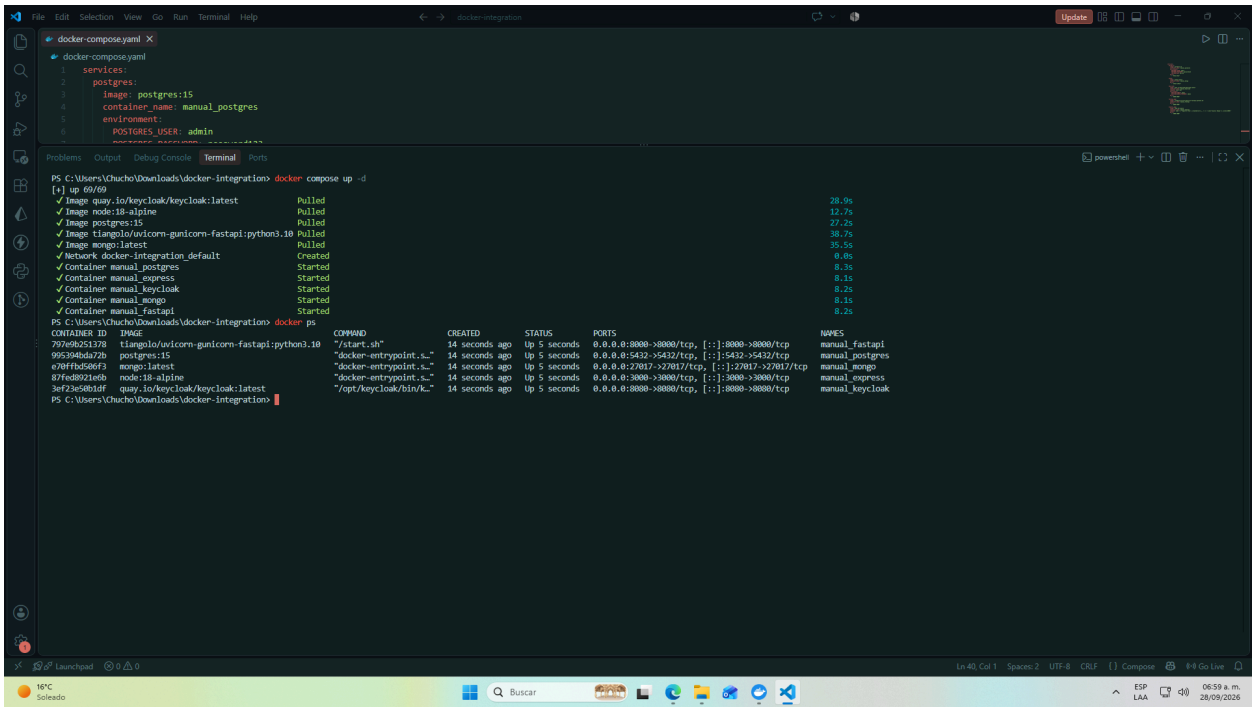
Comandos de Despliegue y Verificación

1. En la misma carpeta donde se creó el archivo docker-compose.yml, ejecutar el comando para levantar los servicios en segundo plano (detached):

   `docker compose up -d`

2. Verificar que todos los contenedores estén corriendo activos mediante el comando:

   `docker ps`


docker-compose.yaml
services:
postgres:
image: postgres:15
container_name: manual_postgres
environment:
POSTGRES_USER: admin
PS C:\Users\Chucho\Downloads\docker-integration> docker compose up -d
[+] up 69/69
✓ Image quay.io/keycloak/keycloak:latest Pulled 28.9s
✓ Image node:18-alpine Pulled 12.7s
✓ Image postgres:15 Pulled 27.2s
✓ Image tiangolo/uvicorn-gunicorn-fastapi:python3.10 Pulled 38.7s
✓ Image mongo:latest Pulled 35.5s
✓ Network docker-integration_default Created 0.0s
✓ Container manual_postgres Started 8.3s
✓ Container manual_express Started 8.1s
✓ Container manual_keycloak Started 8.2s
✓ Container manual_mongo Started 8.1s
✓ Container manual_fastapi Started 8.2s
PS C:\Users\Chucho\Downloads\docker-integration> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
797e9b251378 tiangolo/uvicorn-gunicorn-fastapi:python3.10 "/start.sh" 14 seconds ago Up 5 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp manual_fastapi
995394bda72b postgres:15 "docker-entrypoint.s…" 14 seconds ago Up 5 seconds 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp manual_postgres
e70ffbd506f3 mongo:latest "docker-entrypoint.s…" 14 seconds ago Up 5 seconds 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp manual_mongo
87fed8921e6b node:18-alpine "docker-entrypoint.s…" 14 seconds ago Up 5 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp manual_express
3ef23e50b1df quay.io/keycloak/keycloak:latest "/opt/keycloak/bin/k…" 14 seconds ago Up 5 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp manual_keycloak
PS C:\Users\Chucho\Downloads\docker-integration>

6. Módulo 4: Bases de Datos, Herramientas API y FastAPI

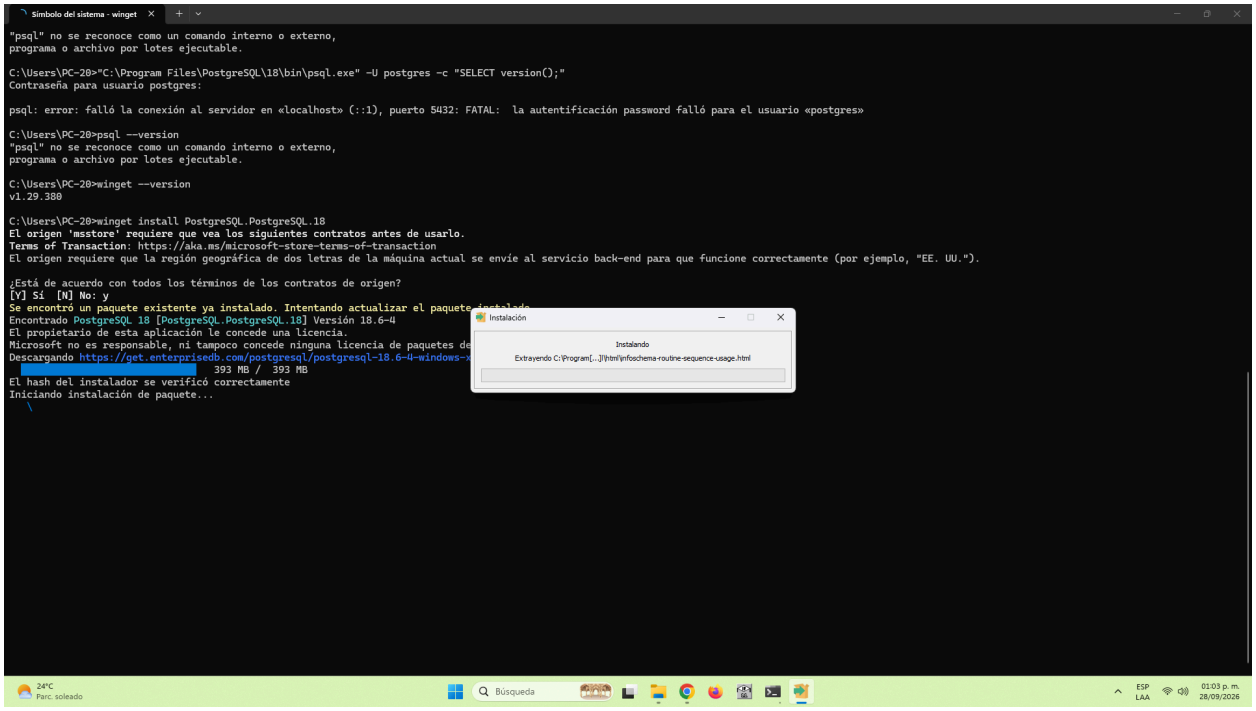
En este módulo prepararemos las herramientas fundamentales de nuestro stack tecnológico. Configuraremos nuestras bases de datos (SQL y NoSQL), instalaremos nuestro cliente de pruebas de API y levantaremos nuestro primer servidor backend para confirmar que todo funciona correctamente.

6.1. Instalación y Comprobación de PostgreSQL

PostgreSQL es uno de los motores de bases de datos relacionales más robustos del mercado. Para su instalación, te recomendamos descargar el instalador oficial desde postgresql.org/download según tu sistema operativo, o utilizar el gestor de paquetes de tu sistema (ej. sudo apt install postgresql en Ubuntu o brew install postgresql en macOS).

Comprobación y prueba de conexión: Una vez instalado y con el servicio en ejecución, abre tu terminal y ejecuta el siguiente comando para probar la conexión directa al servidor y verificar la versión instalada:

Bash
```
psql -U postgres -c "SELECT version();"
```



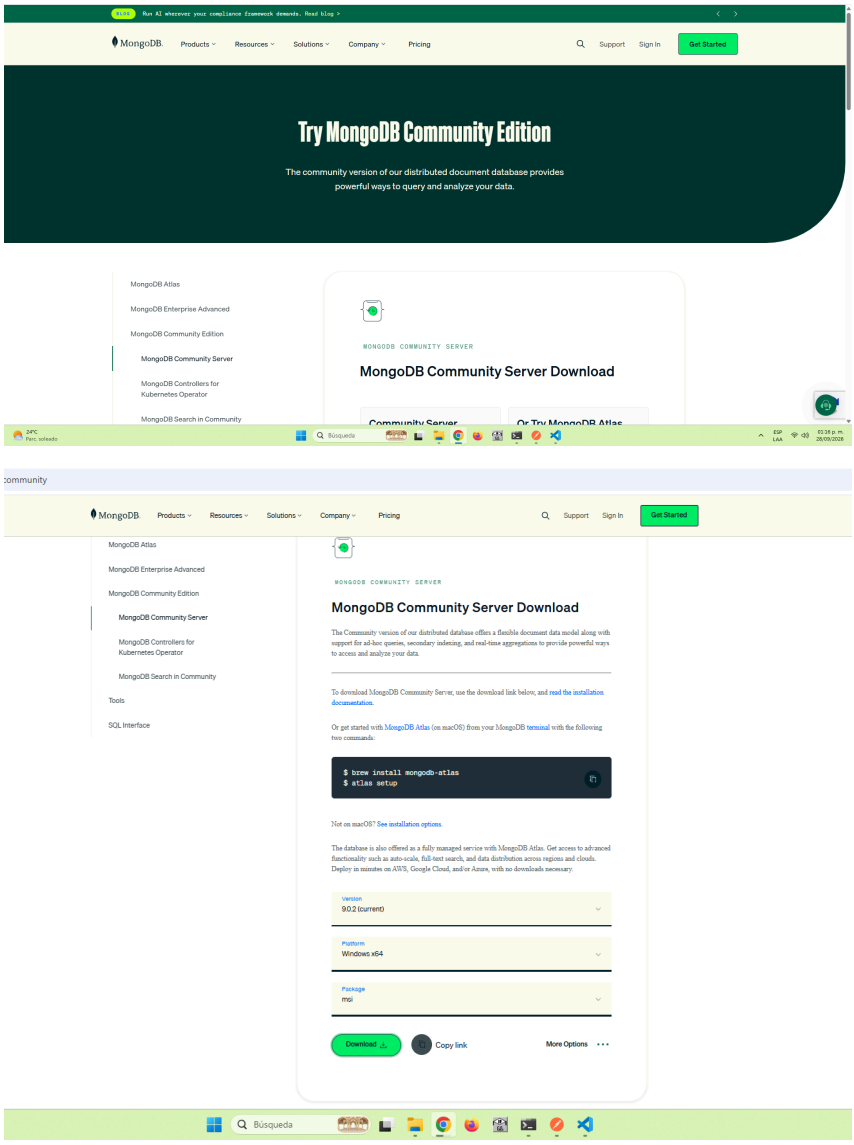

```
¿Está de acuerdo con todos los términos de los contratos de origen?
[Y] Sí  [N] No: y
Se encontró un paquete existente ya instalado. Intentando actualizar el paquete instalado...
Encontrado PostgreSQL 18 [PostgreSQL.PostgreSQL.18] Versión 18.6-4
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://get.enterprisedb.com/postgresql/postgresql-18.6-4-windows-x64.exe
  ██████████████████████████████  393 MB /  393 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente

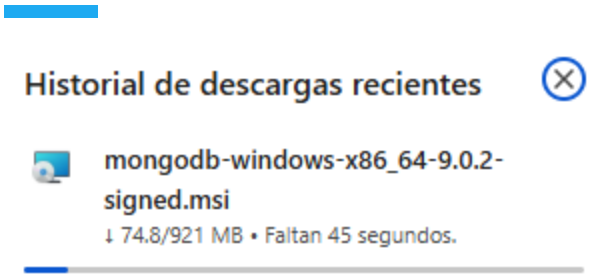
C:\Users\PC-20>
```

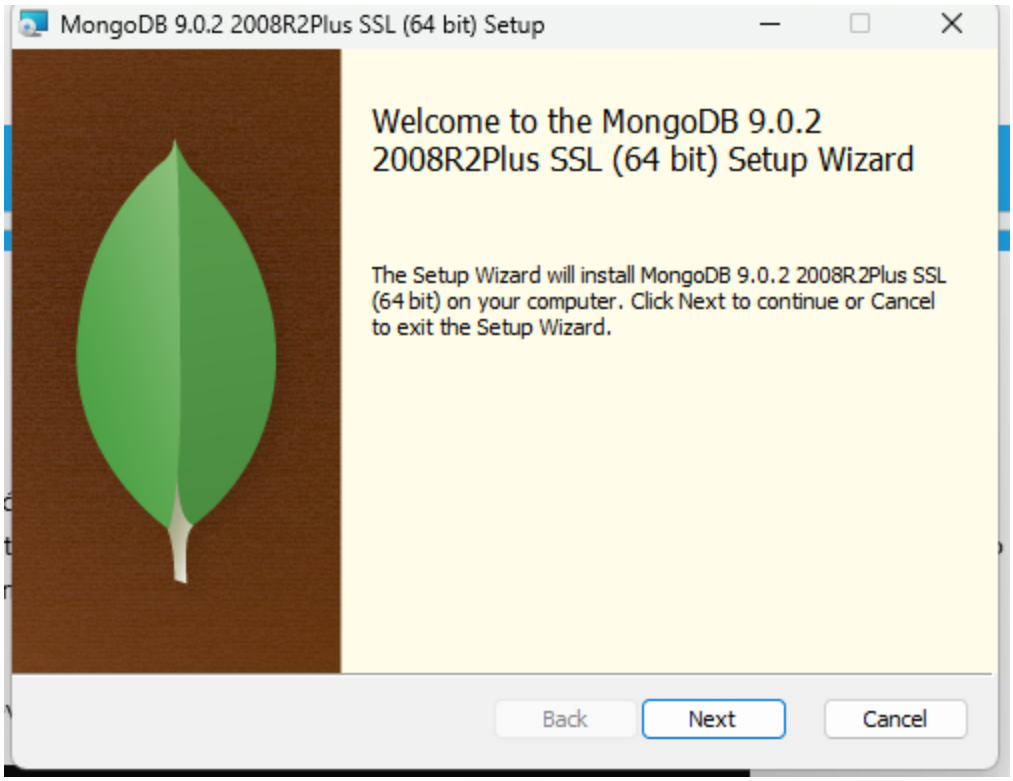
(Nota: Si estás en Linux y te pide autenticación, puedes usar sudo -u postgres psql -c "SELECT version();"). Si la conexión es exitosa, la consola te devolverá la versión de PostgreSQL instalada, confirmando que el motor está listo para recibir consultas.

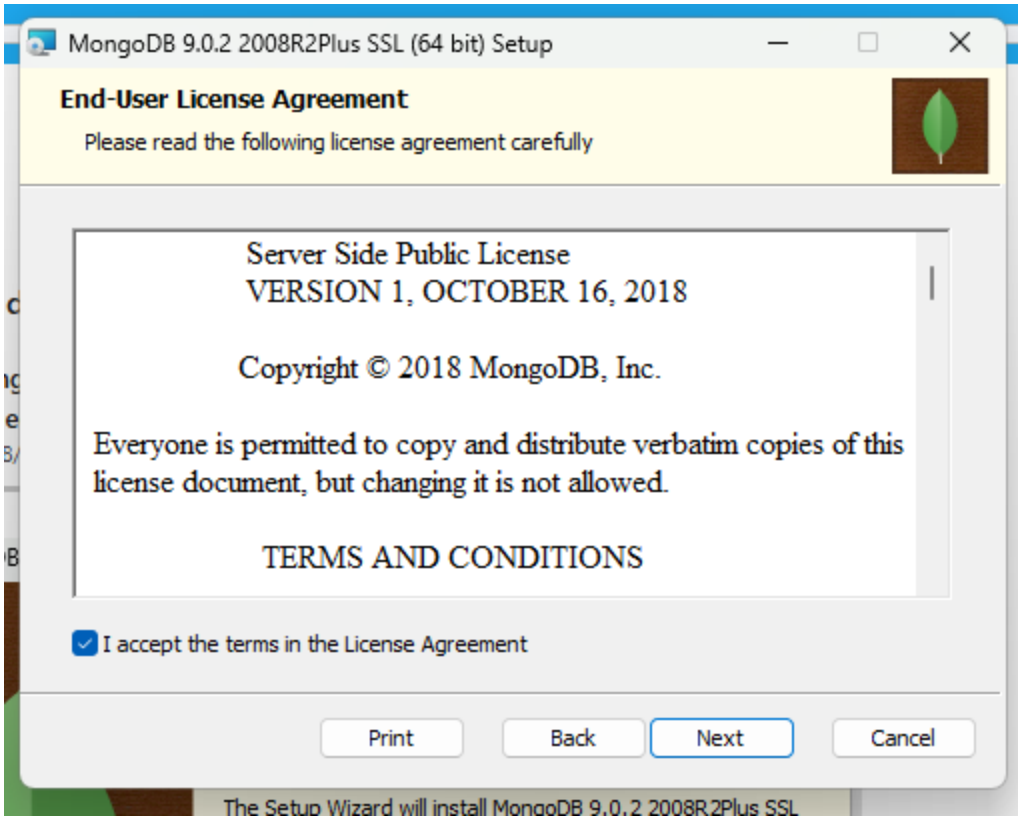
6.2. Instalación y Comprobación de MongoDB

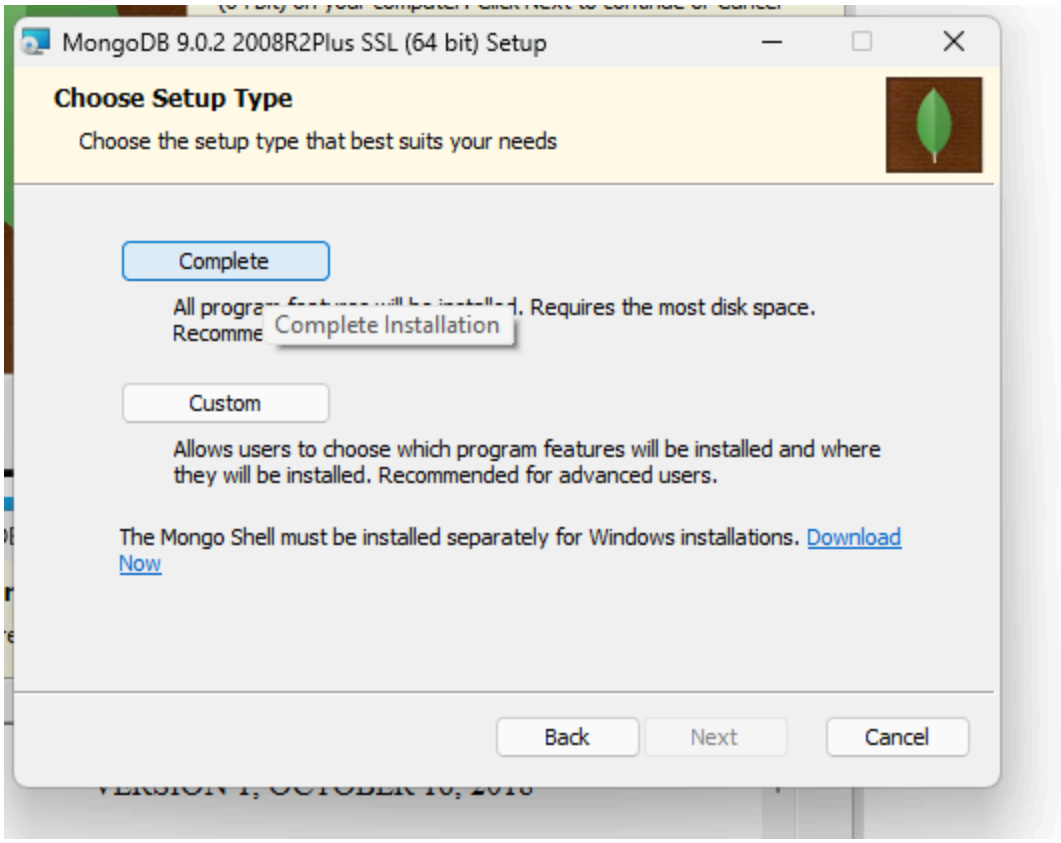
Para manejar datos no estructurados y documentos, utilizaremos MongoDB. Descarga la versión "Community Server" desde el sitio oficial de MongoDB y sigue las instrucciones para tu sistema, asegurándote de instalar también MongoDB Shell (mongosh).

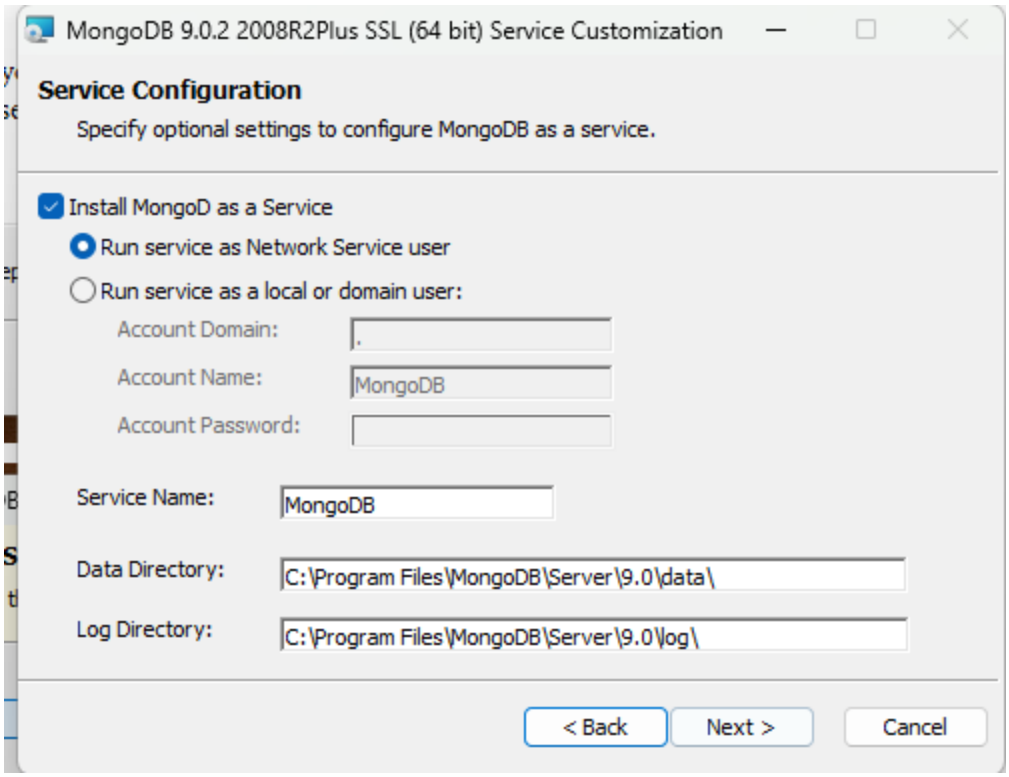


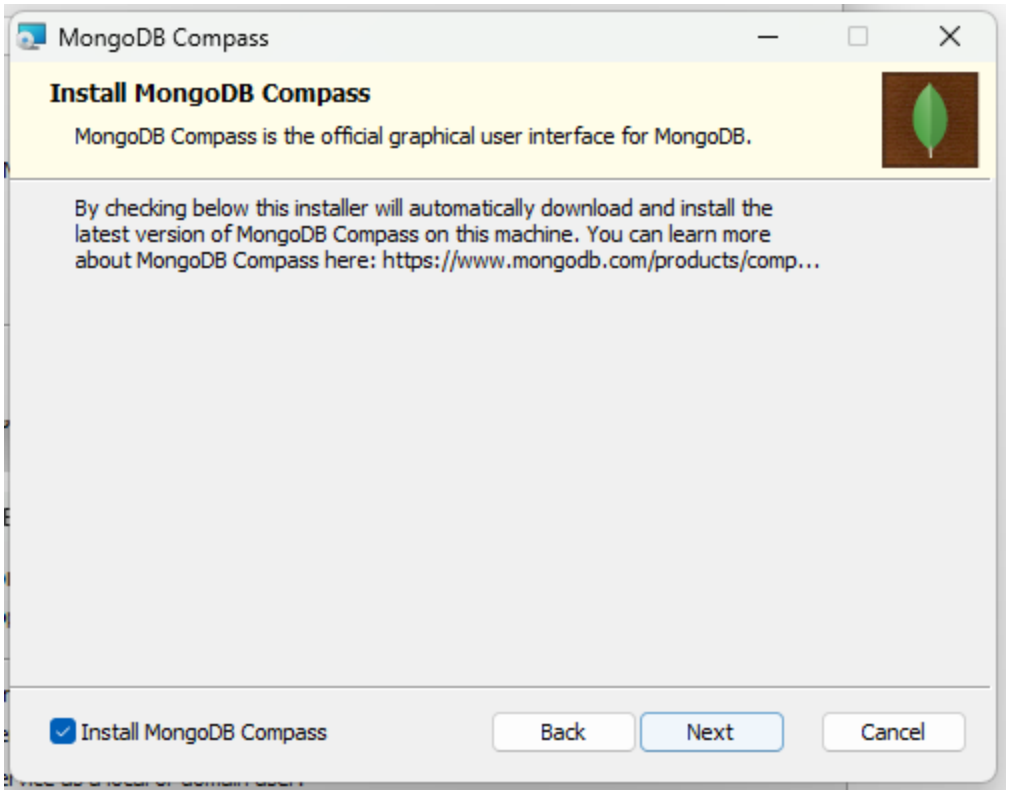

Historial de descargas recientes
mongodb-windows-x86_64-9.0.2-signed.msi
↓ 74.8/921 MB • Faltan 45 segundos.

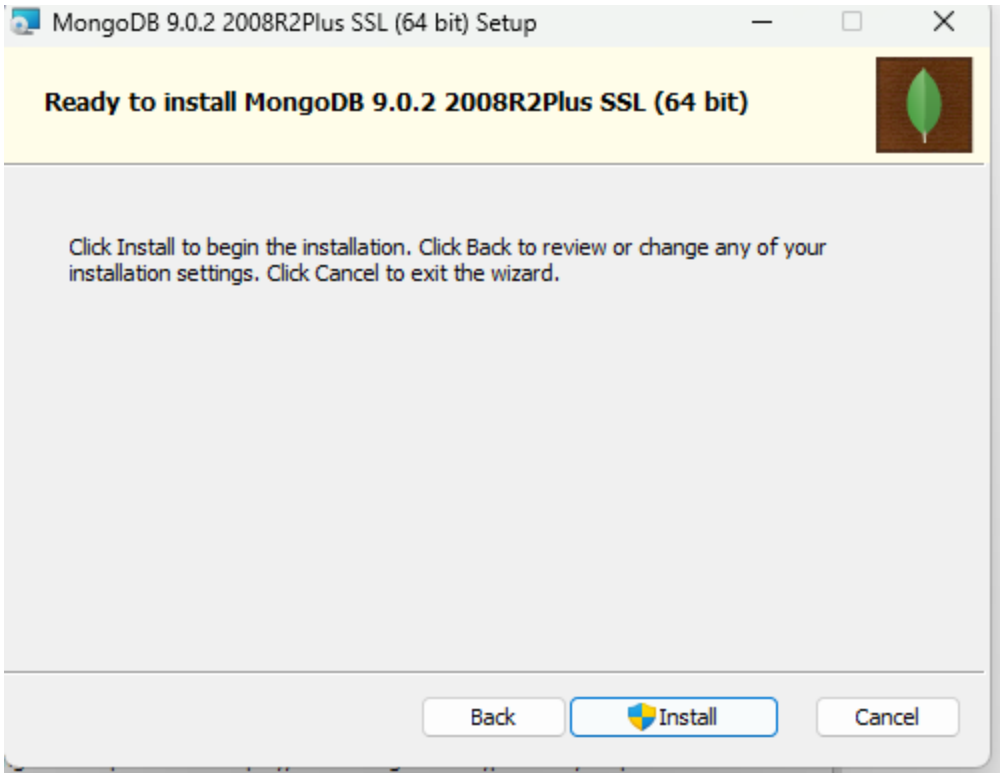

MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup
Welcome to the MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup Wizard
The Setup Wizard will install MongoDB 9.0.2 2008R2Plus SSL (64 bit) on your computer. Click Next to continue or Cancel to exit the Setup Wizard.
Back
Next
Cancel

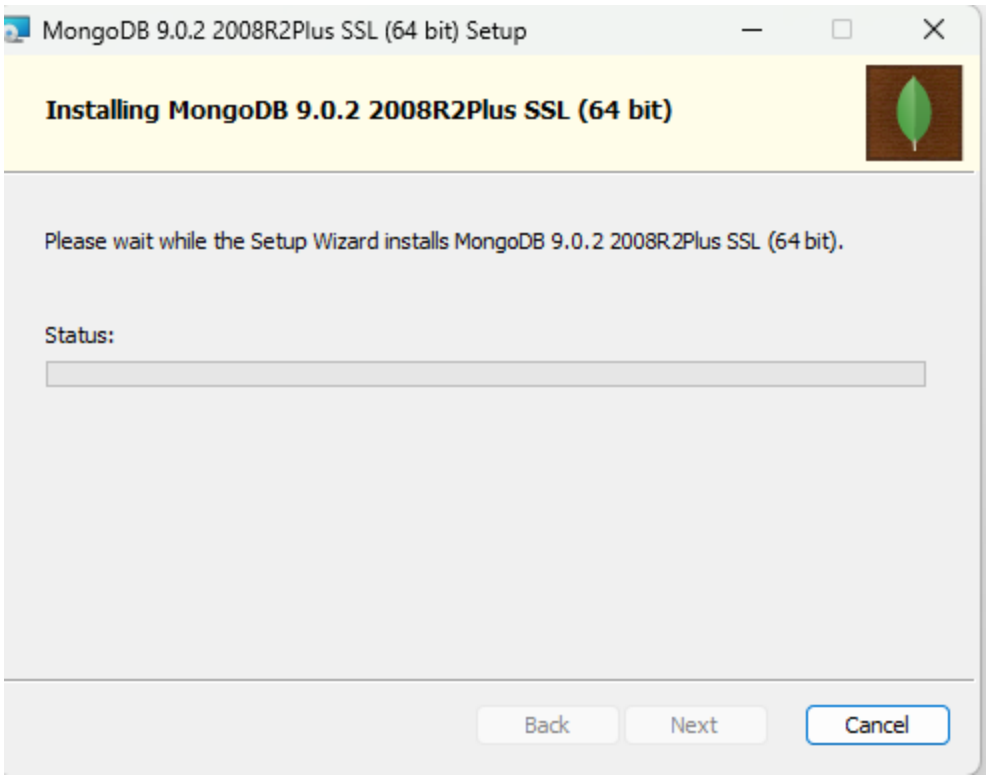

MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup
End-User License Agreement
Please read the following license agreement carefully
Server Side Public License
VERSION 1, OCTOBER 16, 2018
Copyright © 2018 MongoDB, Inc.
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.
TERMS AND CONDITIONS
I accept the terms in the License Agreement
Print
Back
Next
Cancel
The Setup Wizard will install MongoDB 9.0.2 2008R2Plus SSL


MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup
Choose Setup Type
Choose the setup type that best suits your needs
Complete
Complete Installation
Custom
Allows users to choose which program features will be installed and where they will be installed. Recommended for advanced users.
The Mongo Shell must be installed separately for Windows installations. Download Now
Back
Next
Cancel


MongoDB 9.0.2 2008R2Plus SSL (64 bit) Service Customization
Service Configuration
Specify optional settings to configure MongoDB as a service.
Install MongoD as a Service
Run service as Network Service user
Run service as a local or domain user:
Account Domain:
Account Name:
MongoDB
Account Password:
Service Name:
MongoDB
Data Directory:
C:\Program Files\MongoDB\Server\9.0\data\
Log Directory:
C:\Program Files\MongoDB\Server\9.0\log\
< Back
Next >
Cancel

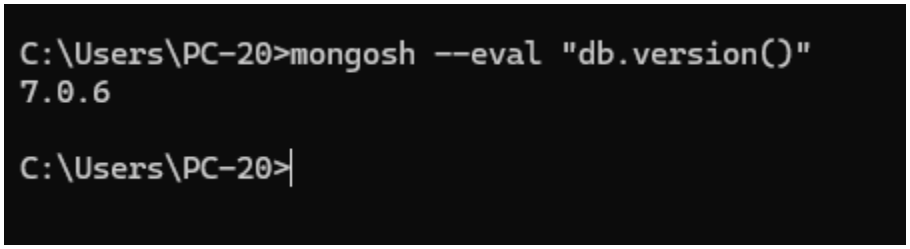

MongoDB Compass
Install MongoDB Compass
MongoDB Compass is the official graphical user interface for MongoDB.
By checking below this installer will automatically download and install the latest version of MongoDB Compass on this machine. You can learn more about MongoDB Compass here: https://www.mongodb.com/products/comp...
Install MongoDB Compass
Back
Next
Cancel


MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup
Ready to install MongoDB 9.0.2 2008R2Plus SSL (64 bit)
Click Install to begin the installation. Click Back to review or change any of your installation settings. Click Cancel to exit the wizard.
Back
Install
Cancel


MongoDB 9.0.2 2008R2Plus SSL (64 bit) Setup
Installing MongoDB 9.0.2 2008R2Plus SSL (64 bit)
Please wait while the Setup Wizard installs MongoDB 9.0.2 2008R2Plus SSL (64 bit).
Status:
Back
Next
Cancel

Comprobación y prueba de conexión: Para asegurarnos de que el demonio de MongoDB (mongod) está corriendo y acepta conexiones, abre la terminal y ejecuta la shell de Mongo con el siguiente comando de prueba:

Bash
mongosh --eval "db.version()"



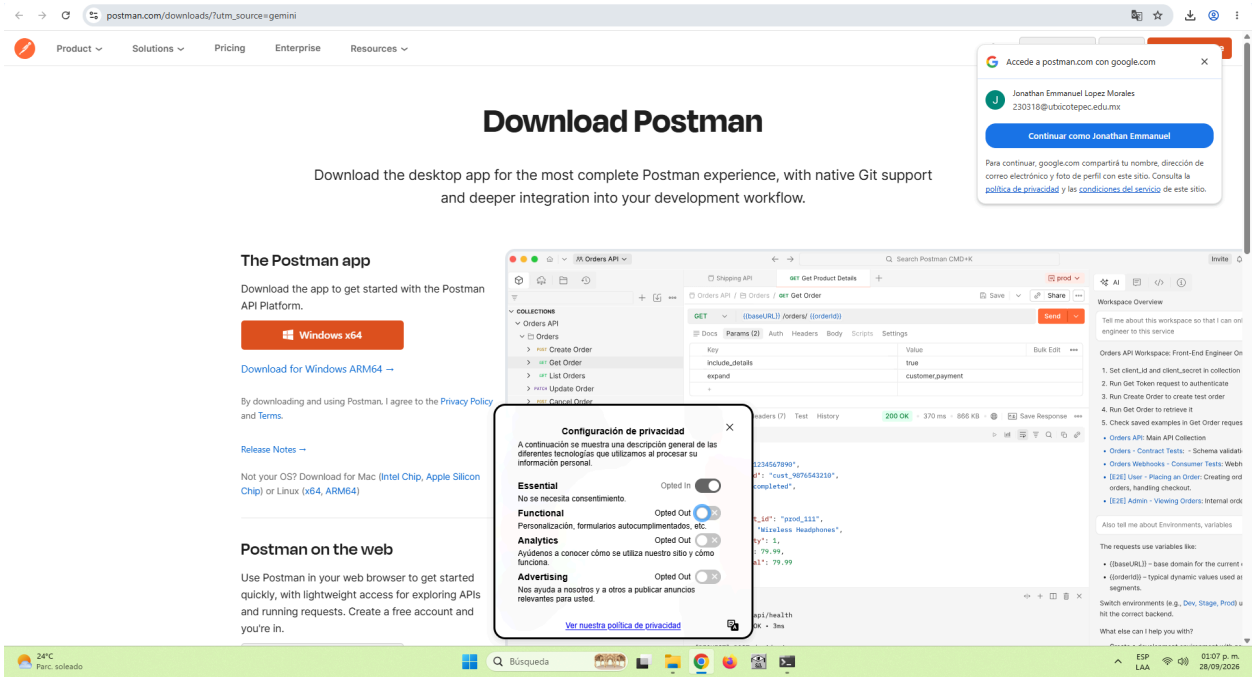
Si el entorno está configurado correctamente, este comando se conectará a la base de datos local y te imprimirá en pantalla la versión actual de MongoDB.

6.3. Instalación del Cliente Postman

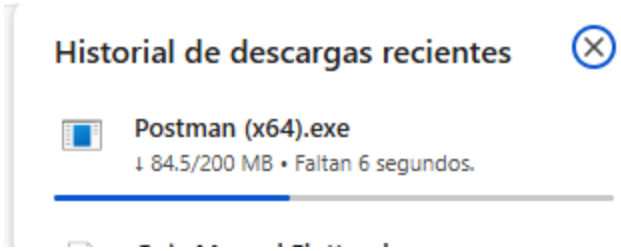
Postman es nuestra herramienta de cabecera para interactuar con las APIs que desarrollemos, permitiéndonos realizar peticiones HTTP de forma gráfica.

Descarga e Instalación:

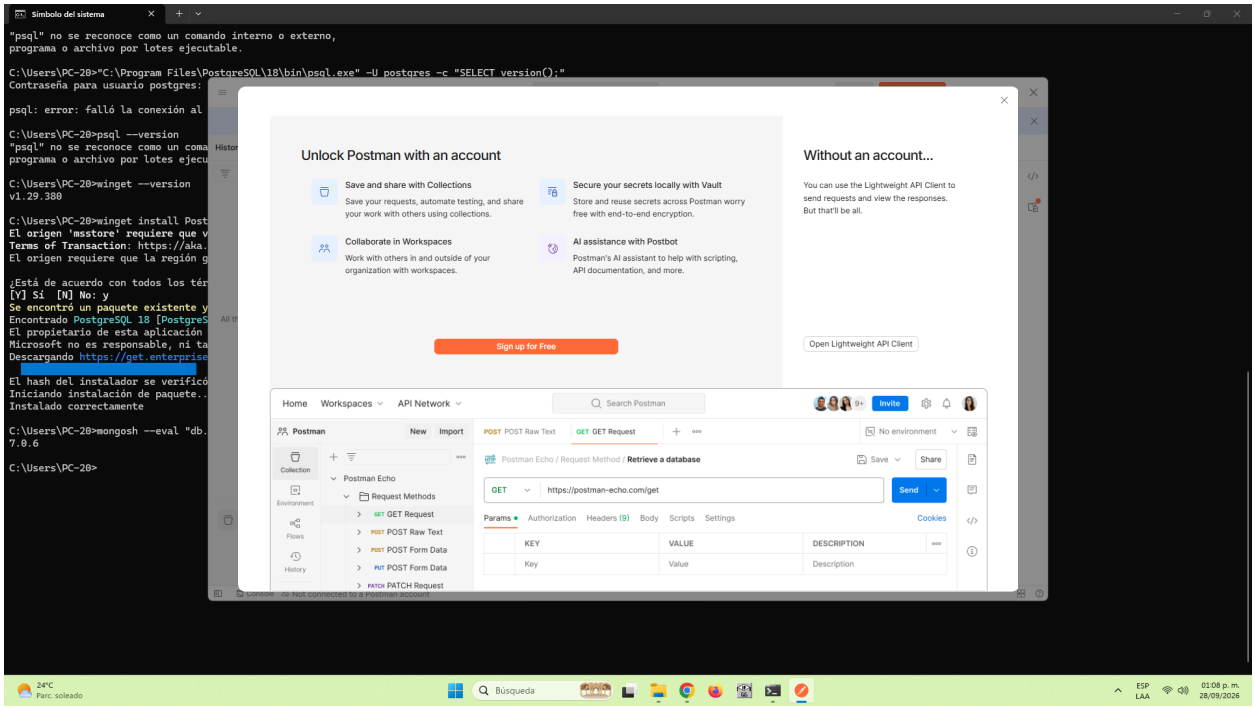
1. Dirígete a la página oficial de descargas: postman.com/downloads/



2. Haz clic en el botón de descarga correspondiente a tu sistema operativo (Windows, macOS o Linux).



3. Ejecuta el archivo descargado para instalar el cliente de escritorio.



Alternativamente, si utilizas un gestor de paquetes por consola, puedes instalarlo rápidamente con:

- Linux (Snap): sudo snap install postman
- macOS (Homebrew): brew install --cask postman
- Windows (Winget): winget install -e --id Postman.Postman

6.4. Entorno Python, Instalación de FastAPI y Documentación Automática (Swagger UI)

FastAPI es un framework moderno y de alto rendimiento. Para utilizarlo, primero debemos aislar nuestras dependencias creando un entorno virtual de Python.

1. Creación y activación del entorno virtual: Abre tu terminal en la carpeta de tu proyecto y ejecuta:

Bash
Crear el entorno virtual
python -m venv venv

Activar el entorno (Linux/macOS)
source venv/bin/activate
Activar el entorno (Windows)
venv\Scripts\activate

```
C:\Users\PC-20>

C:\Users\PC-20>cd Desktop

C:\Users\PC-20\Desktop>mkdir Proyecto

C:\Users\PC-20\Desktop>cd Proyecto

C:\Users\PC-20\Desktop\Proyecto>python -m venv venv

C:\Users\PC-20\Desktop\Proyecto>source venv/bin/activate
"source" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

C:\Users\PC-20\Desktop\Proyecto>venv\Scripts\activate

(venv) C:\Users\PC-20\Desktop\Proyecto>where python
C:\Users\PC-20\Desktop\Proyecto\venv\Scripts\python.exe
C:\Users\PC-20\AppData\Local\Programs\Python\Python313\python.exe
C:\Users\PC-20\AppData\Local\Microsoft\WindowsApps\python.exe

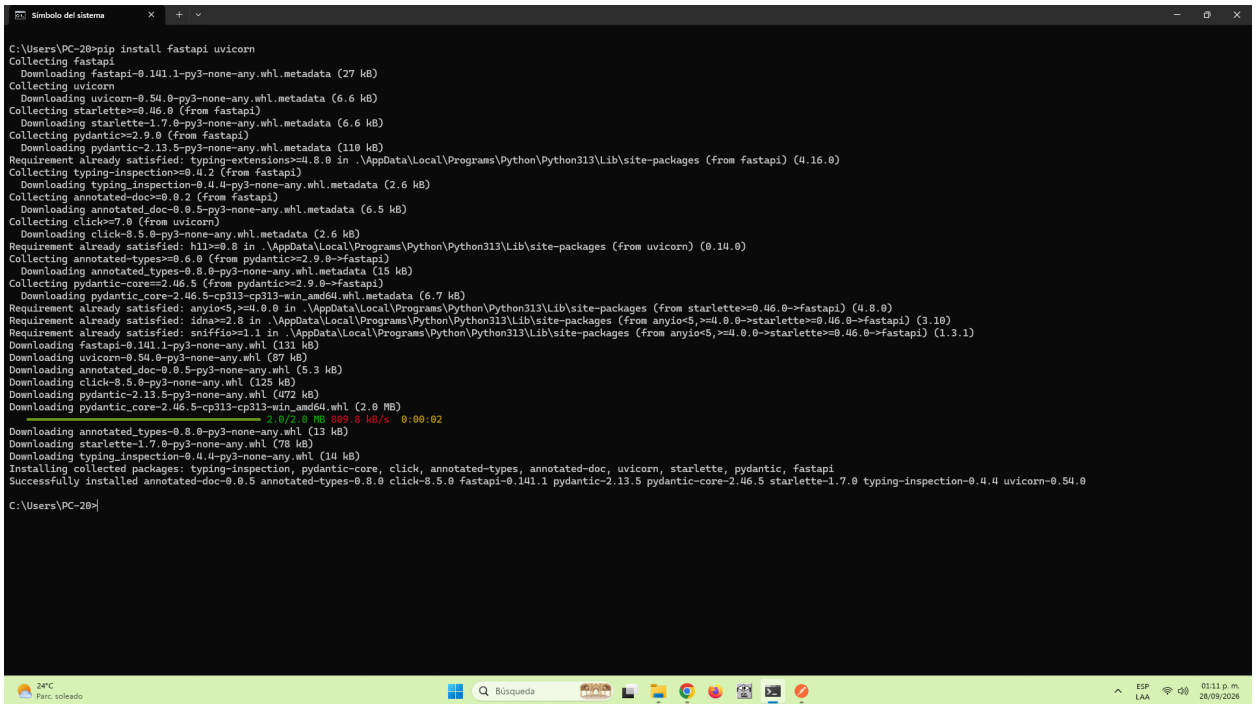
(venv) C:\Users\PC-20\Desktop\Proyecto>python --version
Python 3.13.15

(venv) C:\Users\PC-20\Desktop\Proyecto>
```

2. Instalación de FastAPI y Uvicorn: Con el entorno activado, instalamos el framework y nuestro servidor ASGI (Uvicorn):

Bash

pip install fastapi uvicorn



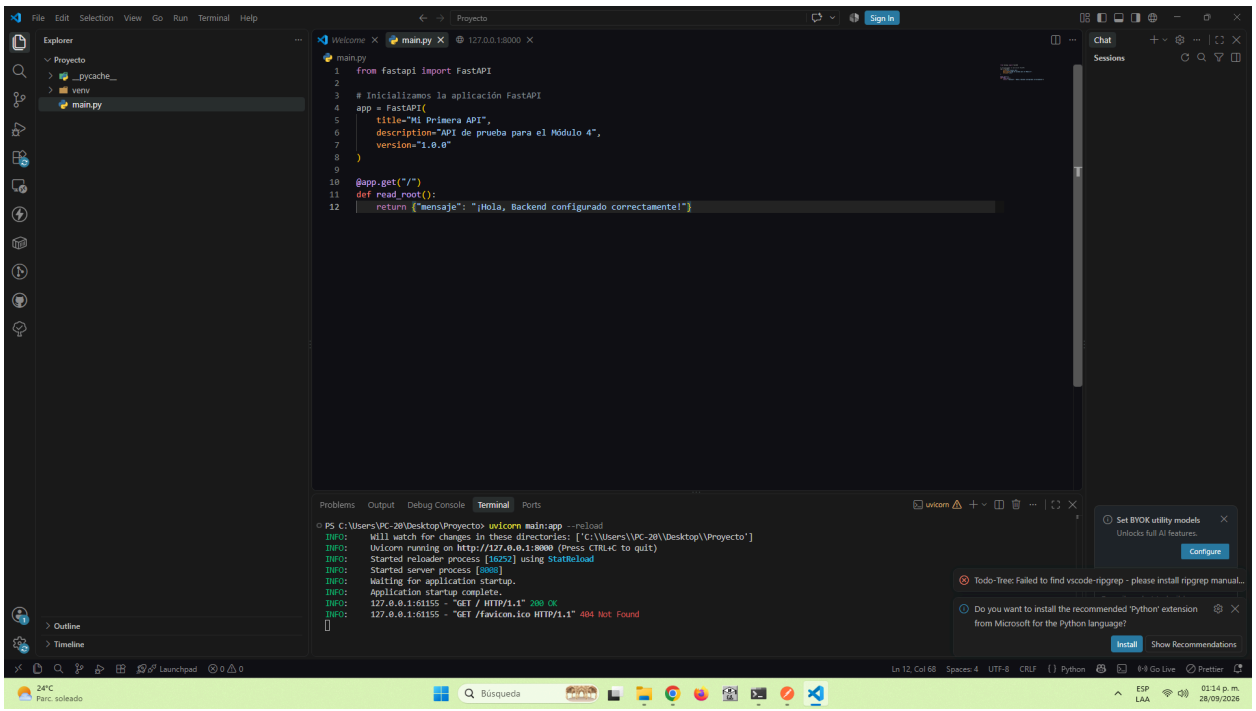
3. Código Básico de FastAPI: Crea un archivo llamado main.py en la raíz de tu proyecto y añade el siguiente código:

Python

```
from fastapi import FastAPI

# Inicializamos la aplicación FastAPI
app = FastAPI(
    title="Mi Primera API",
    description="API de prueba para el Módulo 4",
    version="1.0.0"
)

@app.get("/")
def read_root():
    return {"mensaje": "¡Hola, Backend configurado correctamente!"}
```



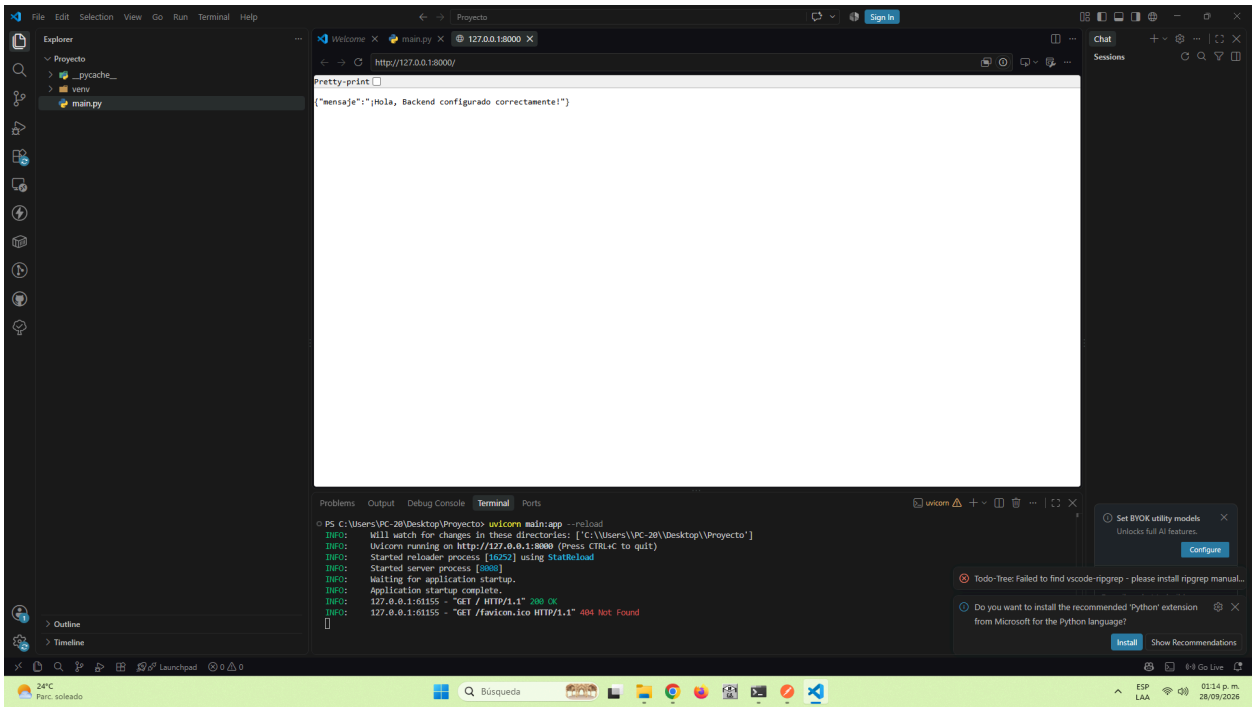
4. Ejecución y visualización de Swagger UI: Para levantar el servidor en modo desarrollo (donde los cambios se recargan automáticamente), ejecuta en tu terminal:

Bash
uvicorn main:app --reload

Una vez que el servidor esté corriendo, abre tu navegador web y dirígete a la siguiente URL para visualizar la documentación automática (Swagger UI) generada por FastAPI:

http://localhost:8000/docs

Desde esta interfaz podrás ver tus endpoints, probar tus peticiones directamente desde el navegador y confirmar que tu entorno Python está 100% operativo.



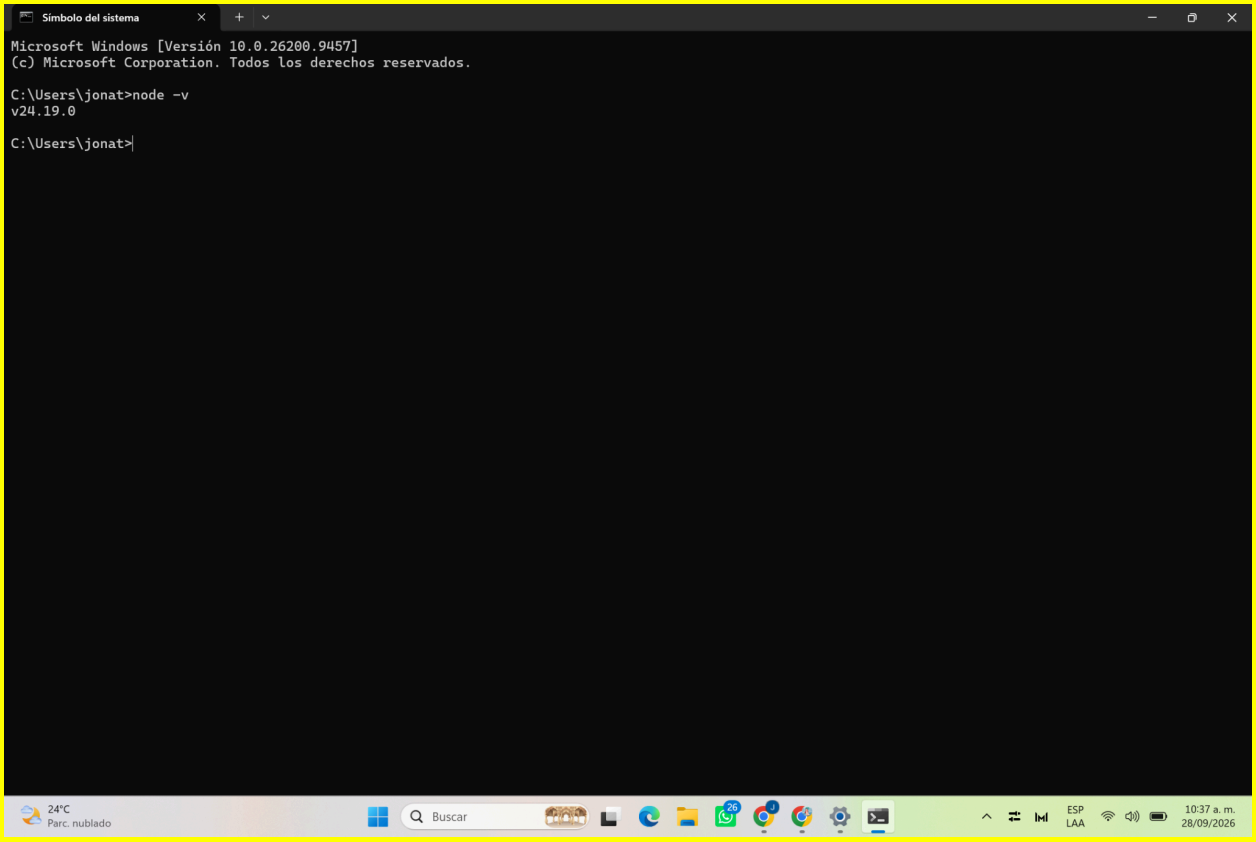
7. Módulo 5: Servicios Backend, Keycloak Y Conexión Flutter

Este módulo tiene como objetivo configurar los servicios básicos de backend mediante Node.js y Express, verificar el funcionamiento de Keycloak y preparar una conexión básica entre una aplicación Flutter y el servidor de autenticación.

7.1 Instalación y Verificación de Node.js y Framework Express

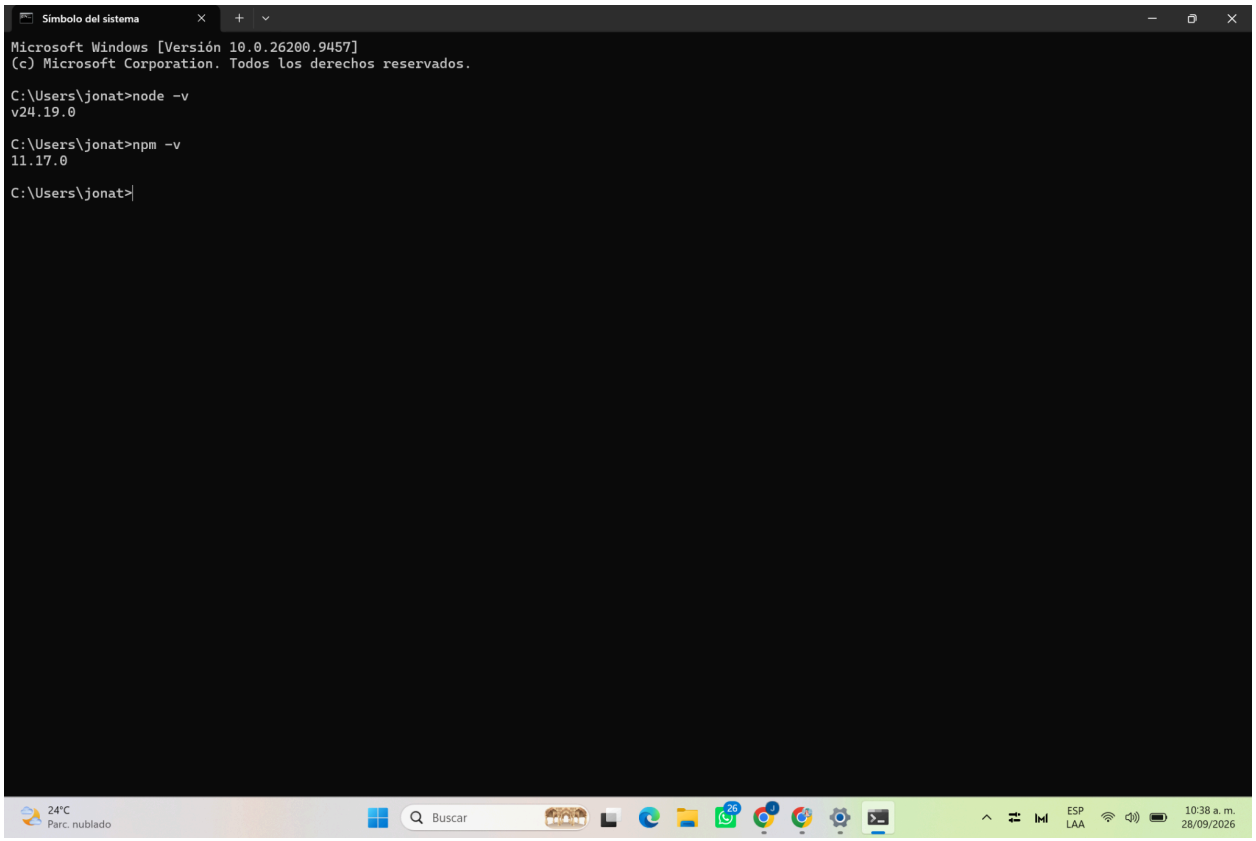
1. Descargar e instalar Node.js en Windows.
2. Una vez finalizada la instalación, abrir una terminal de **PowerShell o CMD**.
3. Comprobar que Node.js se instaló correctamente ejecutando:

```
node -v
```



4. Verificar también la instalación de NPM mediante:

npm -v



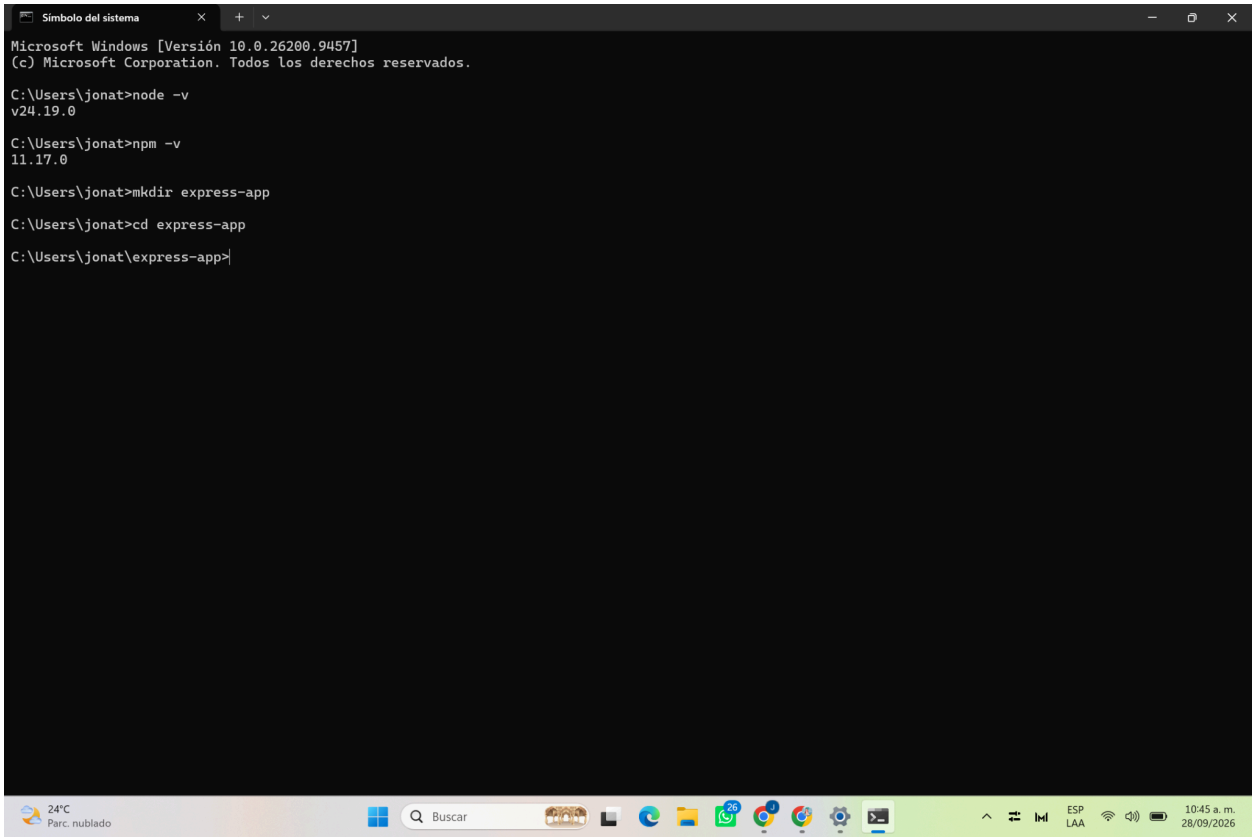
Si ambos comandos muestran un número de versión, la instalación se realizó correctamente.

7.1.2. Creación del proyecto Express

1. Crear una carpeta para el proyecto:

   mkdir express-app

   cd express-app



2. Inicializar el proyecto de Node.js:

   npm init -y

3. Instalar Express:

   npm i express



4. Crear un archivo llamado:

   index.js

5. Agregar el siguiente código:

   const express = require('express');

   const app = express();

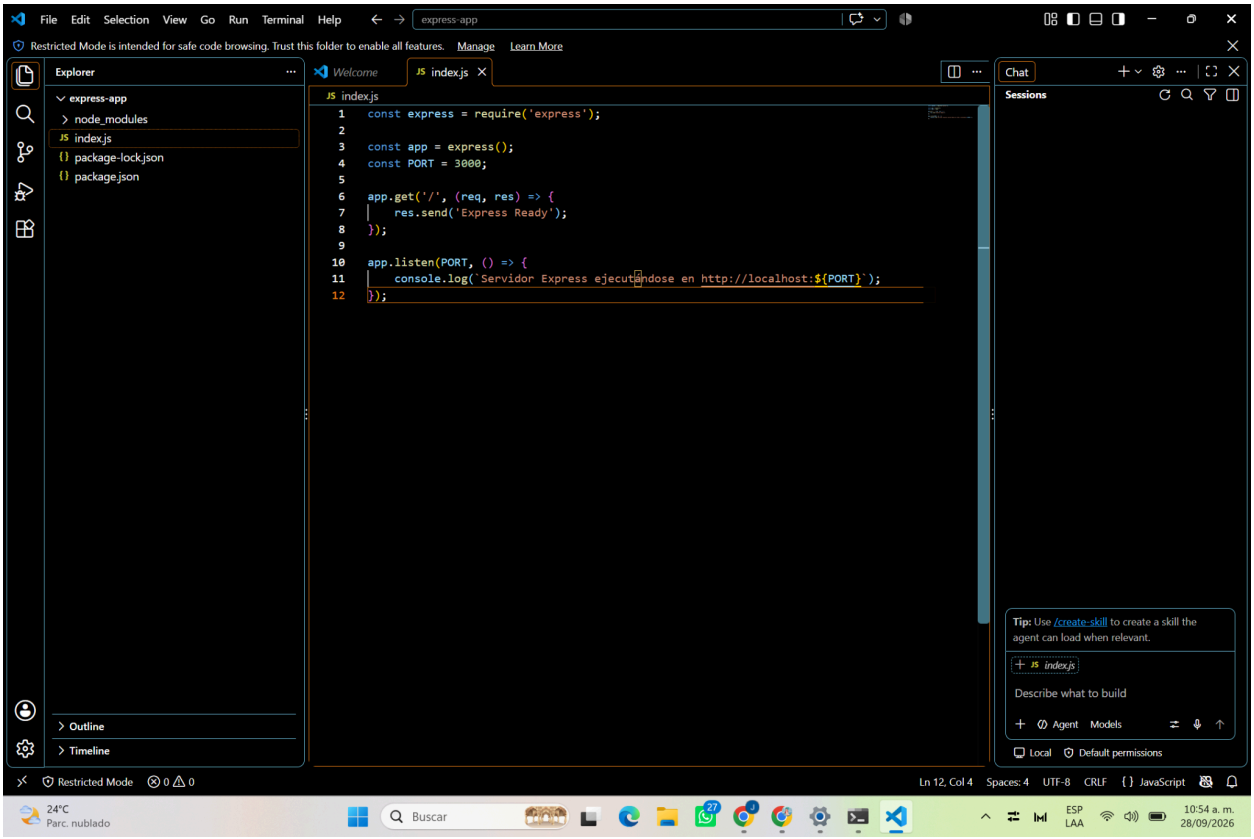
   const PORT = 3000;

   app.get('/', (req, res) => {

   res.send('Express Ready');

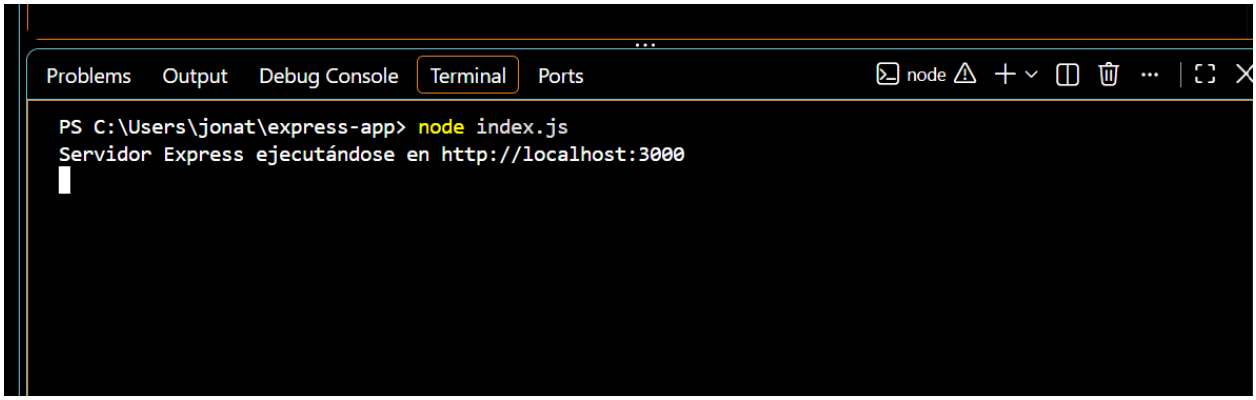
```
});


app.listen(PORT, () => {
    console.log(`Servidor Express ejecutándose en http://localhost:${PORT}`);
});
```



6. Ejecutar el servidor:

   node index.js

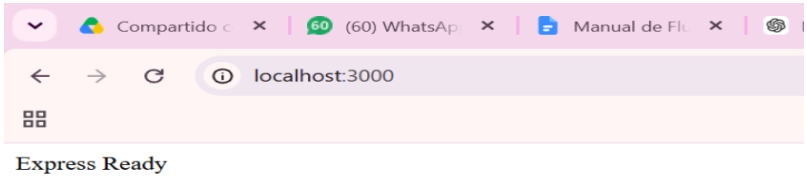


7. Abrir un navegador y acceder a:

   http://localhost:3000

- Debe aparecer el mensaje:

  Express Ready

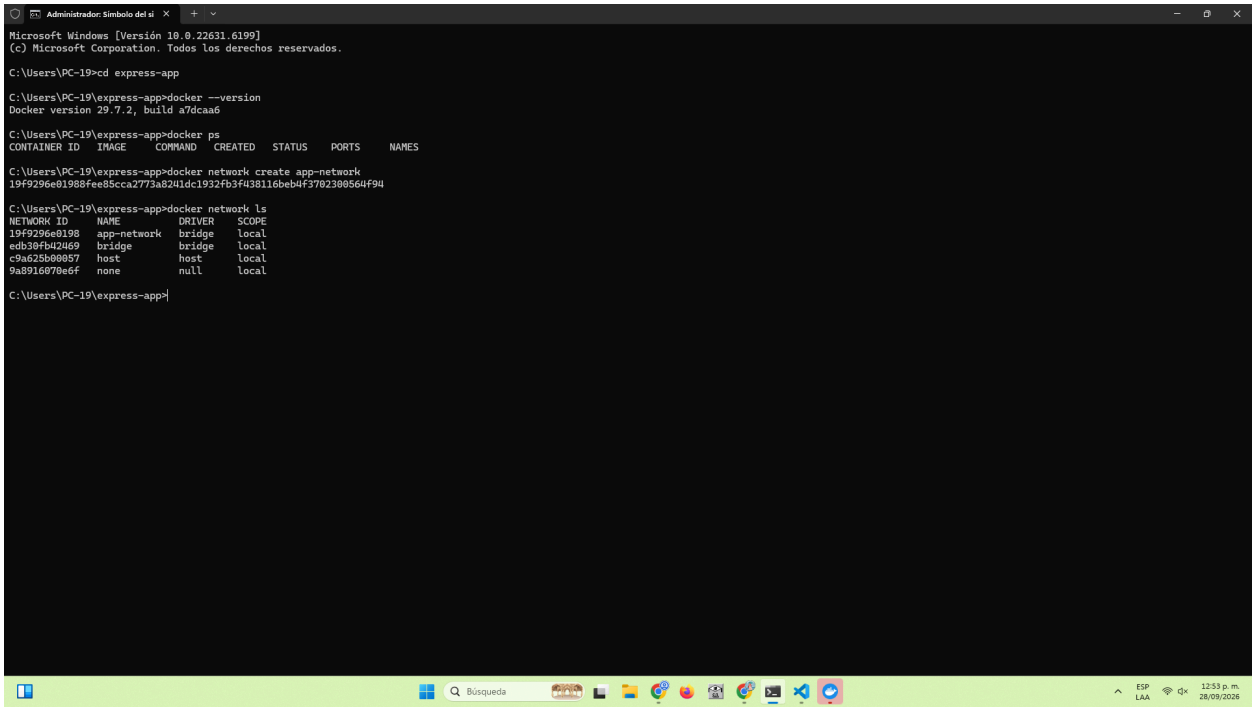


Objetivo de la captura: comprobar que Node.js, NPM y Express están correctamente instalados y que el servidor está funcionando.

7.2 Despliegue e Instalación Base de Keycloak

En esta etapa se realizó el despliegue de **Keycloak** mediante Docker, con el objetivo de disponer de un servidor de identidad para gestionar posteriormente la autenticación y autorización de la aplicación móvil mediante el protocolo **OpenID Connect (OIDC)**.

Como primer requisito se verificó que Docker estuviera correctamente instalado y disponible en el equipo. Para ello se ejecutó el siguiente comando desde la terminal:



Una vez iniciado el contenedor, Keycloak quedó disponible mediante el puerto **8080**, permitiendo acceder a su interfaz web desde el navegador.

La estructura utilizada para el despliegue puede representarse de la siguiente manera:

Equipo local

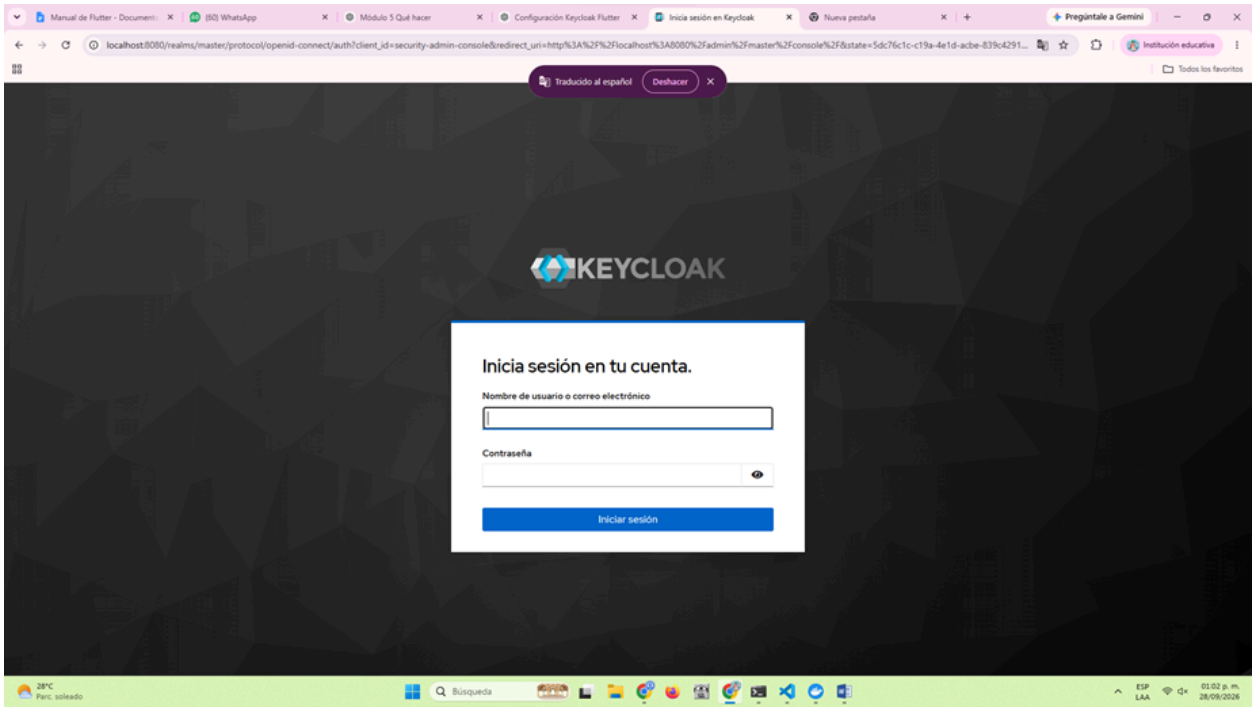
```
|
└── Docker
    |
    └── Contenedor Keycloak
        |
        └── Puerto 8080
```

7.2.1. Verificación de Keycloak

Después del despliegue, se realizó la verificación del funcionamiento de Keycloak mediante el navegador web.

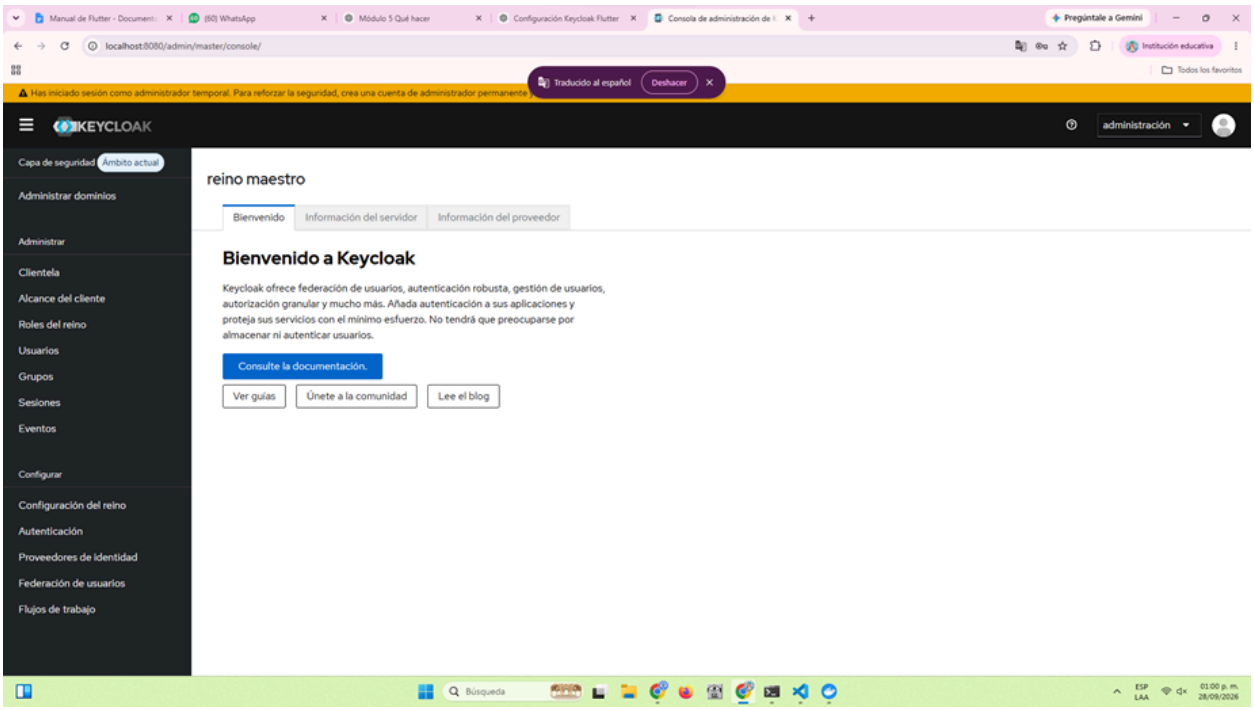
Para comprobar que el servicio estuviera disponible, se accedió a:

http://localhost:8080

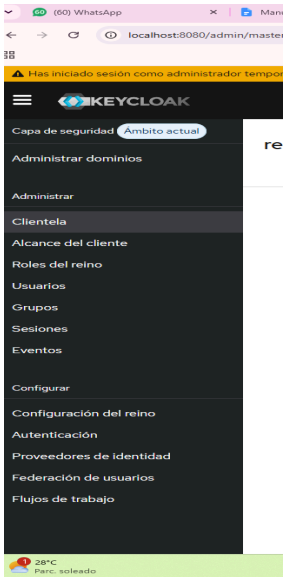


- Posteriormente, se ingresó al panel administrativo mediante:

  http://localhost:8080/admin



- Desde este panel es posible administrar los elementos necesarios para la autenticación, incluyendo **Realms, clientes, usuarios, roles y configuraciones de OpenID Connect**.



La verificación permitió confirmar que el despliegue de Keycloak se realizó correctamente y que el servicio está disponible para continuar con la configuración del cliente OIDC que será utilizado por la aplicación desarrollada en Flutter.

7.3 Configuración Básica de Cliente OIDC para App Móvil en Keycloak

Una vez que Keycloak esté funcionando, vamos a crear el cliente que utilizará Flutter.

En el panel de administración:

Clients → Create client

Configura:

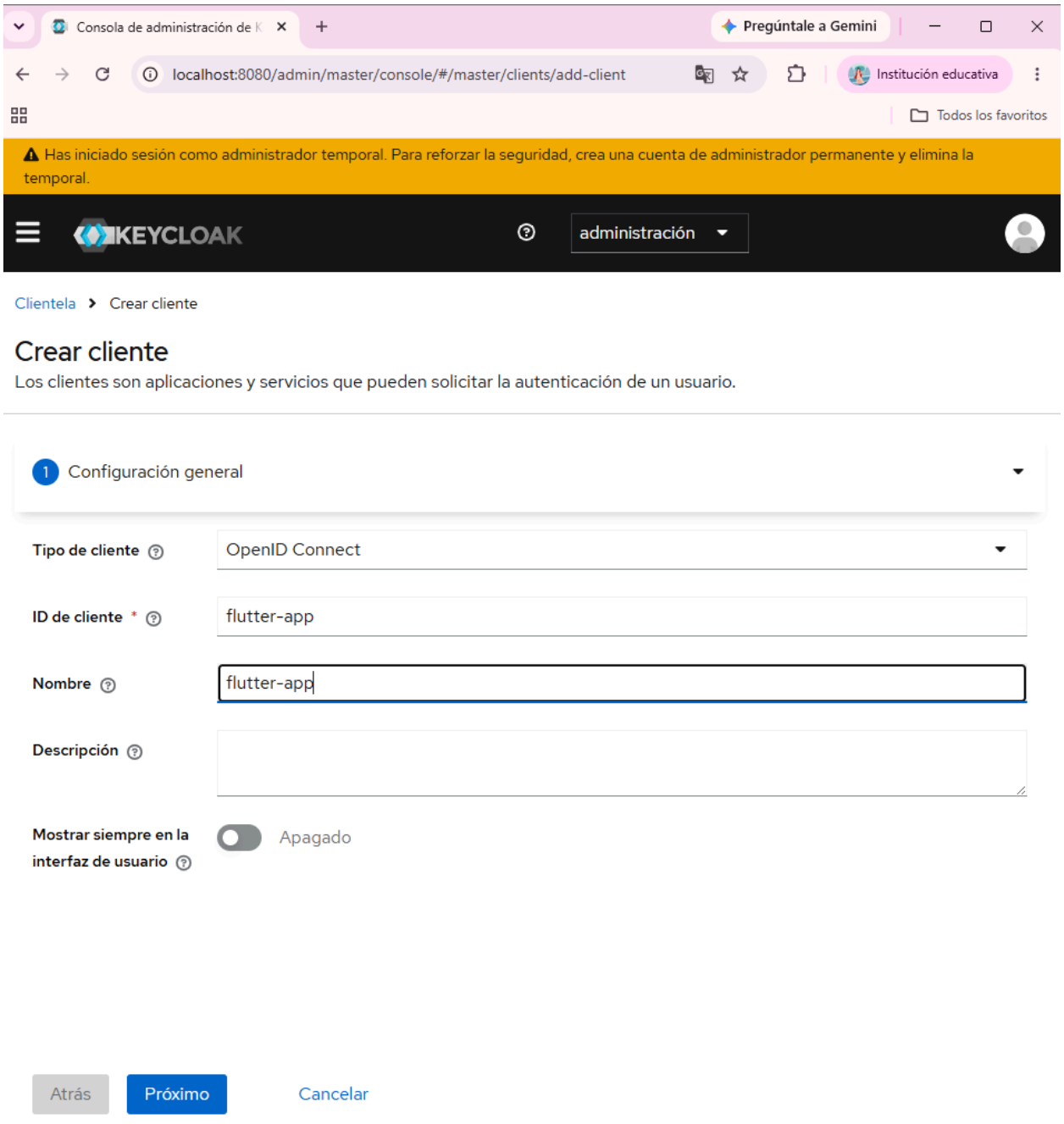
General settings

Client type:

OpenID Connect

Client ID:

nombre-app



Capability config

Para una aplicación móvil utilizaremos OIDC.

Configurar:

Standard flow: ON

Direct access grants: ON

Dependiendo de la versión de Keycloak, algunas opciones pueden aparecer con nombres ligeramente diferentes.

Redirect URI

Para una aplicación móvil es importante configurar correctamente el redirect URI.

Por ejemplo:

com.example.flutterapp://callback

Posteriormente tendremos que utilizar exactamente el mismo esquema en Flutter.

Para desarrollo también puedes dejar preparada una URI como:

http://localhost/*

Guardar

Presionar:

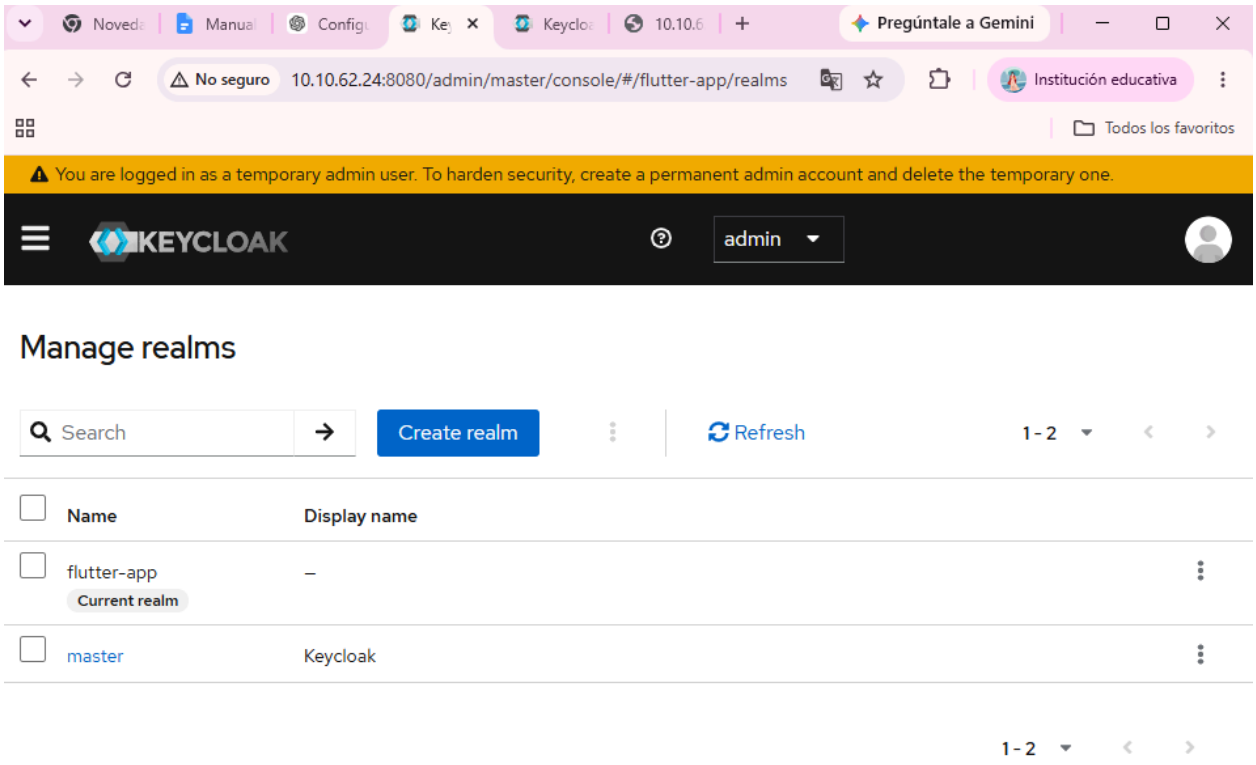
Save

y tendrás creado:

nombre - app

como cliente OIDC.

- Creamos el realm



Entonces la estructura quedaría:

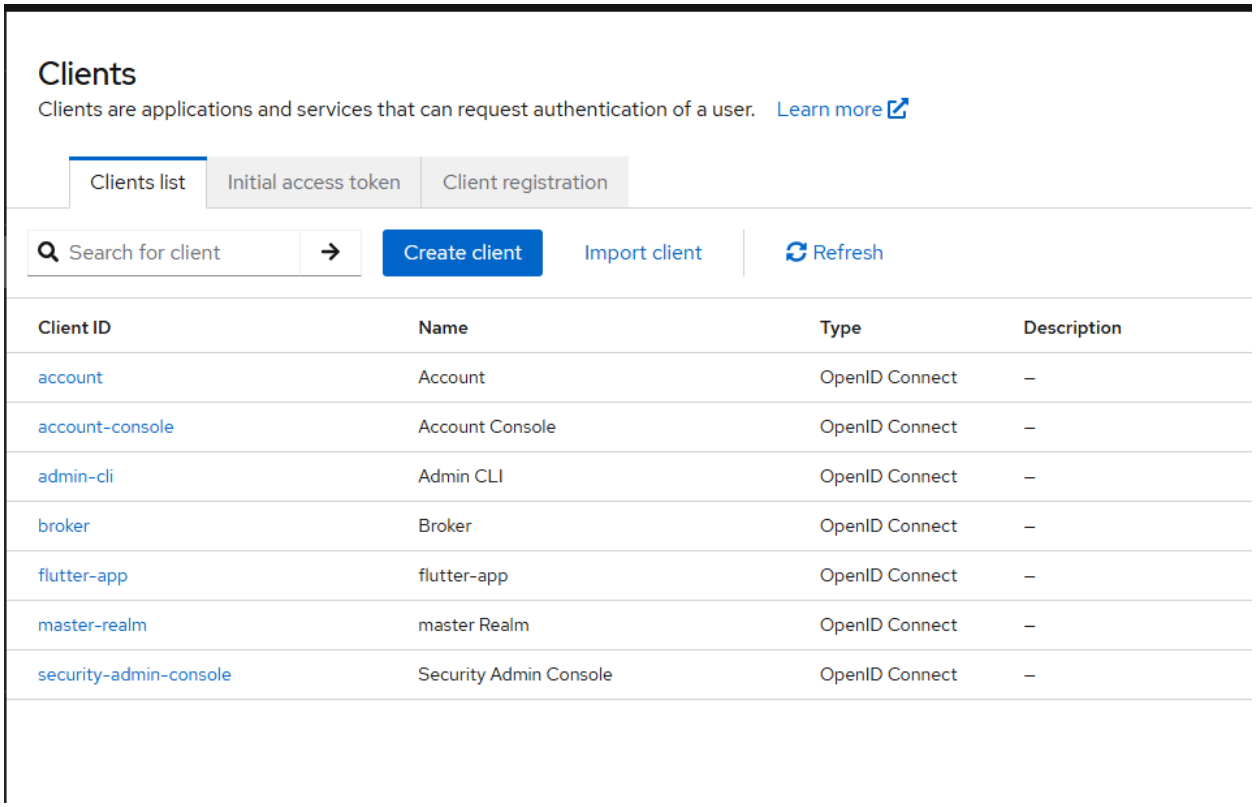
Keycloak

|

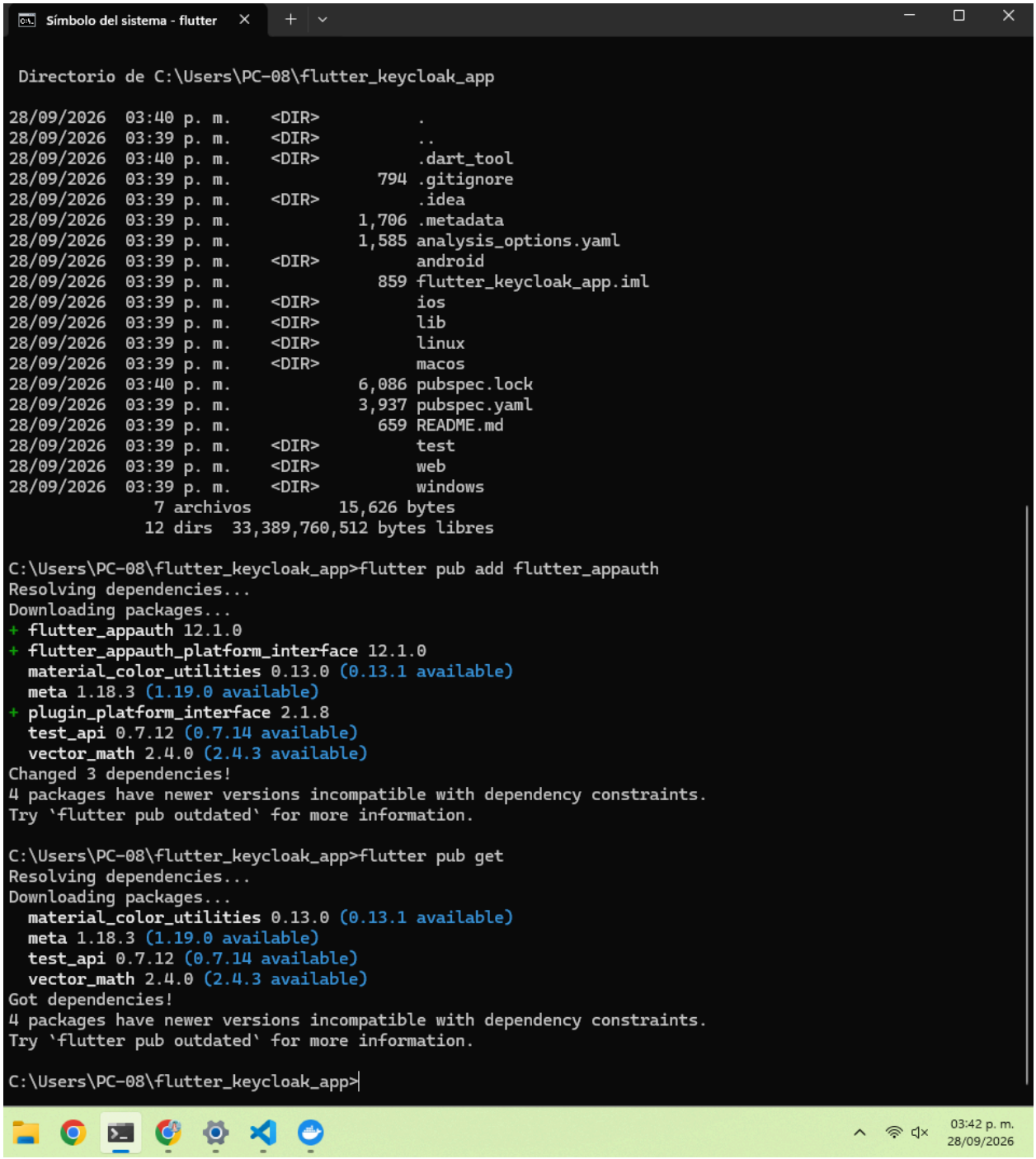
└── flutter-realm

  |

  └── nombre-app


Clients
Clients are applications and services that can request authentication of a user.
Learn more
Clients list
Initial access token
Client registration
Search for client
Create client
Import client
Refresh
Client ID
Name
Type
Description
account
Account
OpenID Connect
–
account-console
Account Console
OpenID Connect
–
admin-cli
Admin CLI
OpenID Connect
–
broker
Broker
OpenID Connect
–
flutter-app
flutter-app
OpenID Connect
–
master-realm
master Realm
OpenID Connect
–
security-admin-console
Security Admin Console
OpenID Connect
–

7.4 Configuración de Conexión Base en Flutter (pubspec.yaml y red 10.0.0.2)



8. Conclusiones y Recomendaciones

8.1. Conclusiones

1. Estandarización Exitosa del Entorno: La separación clara de responsabilidades entre entornos nativos (Windows para Flutter y el Emulador Pixel) y entornos virtualizados/contenedorizados (WSL2 + Docker Desktop) garantiza que todos los desarrolladores del equipo trabajen sobre una arquitectura idéntica, eliminando errores de compatibilidad entre máquinas.
2. Eficiencia en la Orquestación Backend: El uso de Docker Compose V2 permite desplegar la infraestructura completa de persisencia (PostgreSQL, MongoDB) y servicios de aplicación (FastAPI, Express, Keycloak) mediante un único comando, reduciendo tiempos de configuración inicial de días a minutos.
3. Validación Integral de Herramientas: Cada componente documentado en este manual fue instalado y verificado con pruebas de diagnóstico concretas (flutter doctor -v, docker ps, comprobación de endpoints y clientes de base de datos), asegurando un entorno listo para la fase de desarrollo activo.

8.2. Recomendaciones

1. Direccionamiento de Red en Desarrollo Móvil: Mantener siempre presente la regla de red del emulador Android (10.0.2.2 en lugar de localhost). Olvidar esta diferencia es la causa principal de errores de conexión (Connection Refused) al consumir APIs locales desde el dispositivo virtual.
2. Control de Versiones del Entorno: Cualquier adición de un nuevo servicio o cambio de dependencias debe actualizarse de forma inmediata en el archivo docker-compose.yml del repositorio central de GitHub para evitar discrepancias entre los entornos del equipo.
3. Mantenimiento Periódico de Docker: Ejecutar periódicamente el comando docker system prune para limpiar imágenes no utilizadas, contenedores detenidos y caché de compilación que puedan consumir espacio excesivo en el disco asignado a WSL2.